GenAI-Net: A Generative AI Framework for Automated Biomolecular Network Design

Maurice Filo[1†], Nicolò Rossi[1†], Zhou Fang[2†], Mustafa Khammash[1*]

[1]Department of Biosystems Science and Engineering, ETH Zürich, 4056 Basel, Switzerland.

[2]State Key Laboratory of Mathematical Sciences, Academy of Mathematics and Systems Science, Chinese Academy of Sciences, Beijing, 100190, China.

[*]Corresponding author. Email: mustafa.khammash@bsse.ethz.ch [†]These authors contributed equally to this work.

Teaser: GenAI-Net turns biomolecular network design into an AI-driven automated search, discovering diverse solutions across applications.

Biomolecular networks underpin technologies in synthetic biology—from biomanufacturing and metabolic engineering to smart therapeutics and cell-based diagnostics—and provide a mechanistic language for natural and ecological dynamics. Yet designing chemical reaction networks (CRNs) implementing a desired dynamical function remains largely manual: although proposed networks can be checked by simulation, the reverse problem of discovering networks from behavioral specifications is difficult, requiring human insight to navigate vast spaces of topologies and kinetic parameters with nonlinear and possibly stochastic dynamics. Here we introduce GenAI-Net, a generative AI framework that automates CRN design by coupling reaction proposal to simulation-based evaluation defined by a user-specified objective. GenAI-Net efficiently produces topologically diverse solutions across design tasks, including dose response shaping, complex logic gates, classifiers, oscillators, habituation, robust perfect adaptation, and noise reduction in stochastic settings. By turning specifications into families of circuit candidates, GenAI-Net provides a route to programmable biomolecular circuit design and accelerates translation from desired function to implementable mechanisms.

Keywords: chemical reaction networks | automated circuit design | generative AI | reinforcement learning
Corresponding author: mustafa.khammash@bsse.ethz.ch

Introduction

With modern advances in gene editing and DNA synthesis technologies (*1–5*) the rational engineering of living systems has become a practical reality (*6–11*). This capability is poised to transform diverse areas—from precision therapeutics and cell-based diagnostics to sustainable biomanufacturing and new biomaterials—with major implications for human health and the environment (*12, 13*). A central prerequisite is the ability to design biomolecular circuits and reaction networks that implement desired functions while remaining sufficiently compact and robust to operate in complex cellular contexts. Yet this design task remains difficult: biomolecular circuits are inherently nonlinear and often non-modular, and their function emerges from a complex interplay between reaction topology and kinetic parameters. As a result, achieving a desired behavior still typically demands substantial manual iteration and trial-and-error, reflecting the lack of a general, principled design approach.

To date, most biomolecular circuits have been designed by combining human intuition with principles from applied mathematics and classical systems and control theory. This approach enabled seminal synthetic circuits such as the repressilator (*6*), the genetic toggle switch (*7*), and synthetic autoregulation (*8*), which helped launch the field of synthetic biology. More recently, control-theoretic integral feedback has enabled the design of biomolecular circuits that achieve stringent forms of homeostasis (*9, 14–22*). Proportional–Integral–Derivative (PID) control has also been translated into biomolecular circuit architectures to tune transient dynamics and improve performance beyond steady-state regulation (*23–29*). Furthermore, alternative actuation and sensing mechanisms have been explored to achieve similar improvements in dynamical response and noise suppression (*30–34*).

Beyond regulatory feedback motifs, biomolecular networks have been proposed for intricate dynamical responses and biochemical information processing, including minimum realizations (*35*), noise filters (*36*), linear classifiers (*37*), logic circuits (*38–40*), and even artificial neural networks (*41–49*). However, achieving these richer behaviors may require complex networks, and translating such designs to *in vivo* settings remains challenging—motivating automated approaches that can generate diverse candidate mechanisms and expose complexity–performance trade-offs.

Beyond human intuition, biomolecular circuits can also be designed using machine intelligence. Early work (*50*) used exhaustive enumeration to identify adaptive behaviors across all three-node enzyme network topologies. As the number of species grows and enumeration becomes infeasible, learning-based strategies become essential. Other pipelines have leveraged sparse regression (*51*), evolutionary algorithms (*52–55*), tree search (*56*), topological filtering (*57*), Thompson sampling (*58*), and Bayesian optimization (*59*)

to discover biochemical networks with desired functions. More recently, a deep-learning approach based on conditional variational autoencoders was introduced (*60*), using simulations of three-node circuits to guide parameter search and retrieve circuits that exhibit adaptation (*60*). In a complementary direction, a supervised learning approach has been recently applied to large-scale experimental measurements to learn predictive composition-to-function models (*61*). Collectively, these methods demonstrate that machine intelligence can automate and accelerate biomolecular circuit design (*62, 63*); however, existing approaches often remain constrained in either versatility (the range of tasks and network sizes they can handle) or efficiency (the ability to search large topology–parameter spaces without extensive task-specific engineering).

Often, the forward design direction is straightforward: given a proposed circuit, we can usually test whether it performs a desired task (e.g. by simulating its dynamics). The reverse direction—starting from a behavioral specification and discovering a network that realizes it—is far more challenging, because the space of reaction topologies and parameterizations is vast with nonlinear and possibly stochastic dynamics. Here we introduce GenAI-Net, a generative AI framework that tackles this inverse problem by enabling flexible, automated design of chemical reaction networks (CRNs) for a broad range of user-defined tasks. GenAI-Net exploits the asymmetry in difficulty between the forward and reverse problems by placing an AI agent in the loop (Fig. 1). The agent acts as an oracle to automatically propose candidate designs (reverse problem) and pass them to a simulator for performance evaluation (forward problem). The resulting evaluation signals are then used to refine the oracle's accuracy and strengthen the agent's generative capabilities.

A GenAI-Net user starts by specifying a task-level objective—what behavior the CRN should realize—together with the chemical context in which the design must live (Fig. 1, left). The user defines the molecular species and, when relevant, designated inputs and outputs, chooses a kinetic model (e.g., mass-action or Michaelis-Menten, deterministic or stochastic setting), and/or selects a reaction library that constrains which reaction types may be used. With these ingredients, GenAI-Net automatically generates and returns a set of diverse candidate CRNs (Fig. 1, right), enabling the user to compare alternative topologies and parameterizations and select designs that best match performance, simplicity, implementability, and even generalizability (Fig. 1, bottom right).

We apply GenAI-Net to a broad set of design specifications, including dose–response shaping (Hill-function matching, ultrasensitivity, and non-monotonic responses), robust perfect adaptation under setpoint changes and disturbances (both in deterministic models and in stochastic settings where noise reduction is additionally targeted), classifiers that map initial conditions to discrete fates, oscillators with fixed-mean dynamics or input-tunable frequency, logic circuits implementing complex Boolean gates, and habituation/sensitization. In each case, the specification is captured through a simulation-based performance evaluation (e.g., time-domain tracking error, classification loss over initial conditions, frequency-domain objectives, or truth-table accuracy), and GenAI-Net successfully generates diverse CRN designs that meet the desired behaviors. Together, these results position GenAI-Net as a general-purpose engine for programmable molecular circuit design, enabling rapid discovery of diverse reaction motifs directly from high-level specifications. All the code to reproduce the results of this work, and to explore other applications is available in a Github repository (*64*) with an easy-to-use programmatic user interface and detailed documentation and list of generated CRNs (see Data, code and material availability section).

Results

In what follows, we demonstrate how GenAI-Net enables automated, objective-driven discovery of chemical reaction networks across a wide range of dynamical tasks. We first present the core method—how candidate CRNs are represented, generated, and evaluated through simulation, and how this feedback is used to progressively improve design proposals. We then use this framework to produce diverse high-performing networks in multiple application settings, illustrating both the behaviors achieved and the reaction-level motifs that emerge from the search.

Input-Output Chemical Reaction Networks (I/O CRN)

We represent each design as an *input–output chemical reaction network* (I/O CRN): a collection of interacting species connected by reactions, where some quantities are treated as external inputs that modulate reaction rates and one or more species are designated as outputs of interest (to be measured or regulated). Each reaction is defined by its stoichiometry and kinetics. More precisely, a single reaction $\mathcal{R}_j$ over species $(\mathbf{X_1}, \ldots, \mathbf{X_n})$ is written as

$$\mathcal{R}_j: \quad \nu_{1j}\,\mathbf{X_1} + \cdots + \nu_{nj}\,\mathbf{X_n} \xrightarrow{\lambda_j(x,u;\theta)} \nu'_{1j}\,\mathbf{X_1} + \cdots + \nu'_{nj}\,\mathbf{X_n},$$

where ν's $\in \mathbb{N}_0$ are stoichiometric coefficients and $\lambda_j(x,u;\theta)$ is the reaction's *propensity* (rate), which depends on the current state x (species concentration or molecular counts), optional external inputs u that modulate reaction rates, and kinetic parameters θ. Each time this reaction fires, the system state changes by the net stoichiometric update $\Delta_j = \nu'_j - \nu_j$. For a network with m reactions, stacking these Δ_j vectors as columns yields the stoichiometric matrix $S \in \mathbb{Z}^{n\times m}$, and stacking the propensities yields $\lambda(x,u;\theta) \in \mathbb{R}^m_{\geq 0}$. In the deterministic setting, the resulting dynamics are $\dot{x} = S\lambda(x,u;\theta)$; in the stochastic setting, the same S and λ define a continuous-time Markov jump process described by the chemical master equation (CME), typically simulated with the Gillespie algorithm (*65*). An I/O CRN is simply such a collection of reactions, together with a designation of which signals serve as inputs u and which species are treated as outputs.

The Reinforcement Learning (RL) Loop

The RL loop (Fig. 2A) begins when the user specifies (i) a starter I/O CRN and (ii) the maximum number of reactions m that may be appended to complete the design. Together, these choices define the start of the construction process (the starter I/O CRN) and its endpoint (a completed I/O CRN obtained after m reaction additions).

At the core of GenAI-Net is an *environment* (top center of the construction loop in Fig. 2A) whose *state* is the current I/O CRN, which may be incomplete. Initially, the state is exactly the user-provided starter I/O CRN. As the loop unfolds, the state evolves through a discrete, combinatorial *state space* consisting of all CRNs obtainable by appending up to m reactions from the chosen reaction library, together with their associated kinetic parameterizations and any permitted input-modulation assignments. Thus, each state encodes the reactions selected so far (and how they are parameterized) while preserving the starter I/O CRN's species and any designated input/output roles.

Because a CRN state is naturally expressed as a human-readable list of reactions, GenAI-Net includes a *translator* (center left of the construction loop in Fig. 2A) that converts the current I/O CRN into a fixed-format, agent-interpretable representation. Concretely, the translator maps the current set of selected reactions into a structured encoding that, for each library reaction, records whether it is present, its kinetic parameter value(s), and which input channels (if any) modulate its propensity. This translation standardizes variable-sized CRN descriptions into a consistent representation that the policy can condition on, while maintaining a direct correspondence back to reaction-level semantics.

Given the translated state, the *agent* (bottom center of the construction loop in Fig. 2A) selects the next reaction to add. The *action space* is defined by the reaction library: each action corresponds to choosing a reaction identity (and, when relevant, its parameter values and input modulation choices), subject to constraints such as masking reactions already present. The agent's decision is governed by a learned *policy*. Importantly, although the agent consumes the translated encoding, it produces its decision in a *human-interpretable* form—for example, in Fig. 2A (under proposed reaction) the agent selects the highlighted reaction (shown in red) to append next.

A lightweight *stepper* block (center right of the construction loop in Fig. 2A) then applies this action by appending the proposed reaction to the existing CRN, producing the next environment state; in the running example, the stepper augments the template by adding the red reaction to yield the updated I/O CRN shown in Fig. 2A (under updated I/O CRN). This construction loop repeats for m steps (or until a termination condition is met), yielding a *complete* I/O CRN. Once the I/O CRN is complete, GenAI-Net switches from the outer construction loop to an inner *evaluation-and-training* loop. The completed CRN is passed to a *simulator*, its trajectories are scored by the task objective to produce a scalar *loss*, and this loss provides the learning signal used to update the policy network of the agent (Fig. 2B). For a more detailed description of the various components in the RL loop, see Methods.

GenAI-Net generates networks with target dose-responses

We first exemplify GenAI-Net for generating networks with target dose-responses. This task is prevalent in biological studies, aiming to identify the underlying biochemical mechanisms given experimentally measured dose-response curves or generate networks exhibiting such responses for synthetic-biology applications. This problem is often hindered by the vast search space over both network structure and kinetic parameters, making exhaustive search impractical. For instance, we consider a system consisting of three chemical species and four starting reactions as shown in Fig. 3A, where the input signal regulates the production rate of one species, and the remaining degradation reactions (one for each reaction) represent the dilution process due to cell growth. Allowing up to five more doubly bimolecular mass-action reactions yields more than 3×10^7 possible networks (i.e., the number of combinations of 87 candidate reactions taken 5 at a time), and this space expands further when accounting for choices of reaction rate constants.

Despite these challenges, GenAI-Net can generate a diverse set of networks that realize target dose–response curves, including Hill-type, ultra-sensitive, and non-monotonic behaviors (see Fig. 3B–D). As shown, for each target curve, GenAI-Net identifies more than 40 topologically distinct networks whose responses closely match the desired profile. This diversity and precision arise from the balance of the loss and entropy terms of the network in the objective function. In one representative training procedure (bottom panel of Fig. 3C) the entropy term (red) drops quickly when the loss (black) decays rapidly—most notably during the first and third periods separated by dashed lines—indicating strong exploitation to select high-performing I/O CRNs from previously sampled networks. In other periods, when the loss plateaus, the entropy gradually increases, signaling exploration for alternative topologies with lower loss and thereby yielding diverse, high-quality networks. In addition to capturing the target dose response, the generated networks exhibit smooth, fast-converging transient dynamics (second panel in the first row of Fig. 3B). This is encouraged by the loss-function design (see Methods), which evaluates trajectory-level mismatches between simulated dynamics and target values over time (right panel of Fig. 3A).

The generated networks also exhibit recurring topological patterns, highlighting specific design principles. In particular, several reactions are frequently selected by high-performing networks; whereas others are consistently avoided (see the bottom-right panel of Fig. 3B for the Hill-type response). By grouping networks according to topological similarity using the Louvain method (bottom-middle panel of Fig. 3B), we identify several representative topologies across the resulting clusters, as shown in the top-right panel of Fig. 3B. Similar patterns arise in the other dose-response examples, with representative networks shown in Fig. 3C–D.

The number of appended reactions is a key design knob for navigating the trade-off between topological simplicity and achievable performance. Allowing more appended reactions expands the space of candidate mechanisms and can improve the performance of the top I/O CRNs (bottom-left panel of Fig. 3B), with the best-performing network improving steadily as the reaction budget increases.

This flexibility enables users to select a reaction budget that matches the desired balance between compactness and functional precision, depending on the requirements of the application.

GenAI-Net discovers robust perfect adaptation circuits for setpoint tracking and disturbance rejection

One of the fundamental tasks of synthetic biomolecular feedback controllers (*66*) is to maintain homeostasis, a property with broad impact in bioproduction, metabolic engineering, and cell-based therapies, where many diseases arise from homeostatic failure (*67, 68*). Robust perfect adaptation (RPA) (*69–71*) is a stringent form of homeostasis: it enforces exact steady-state regulation of a target variable to a prescribed setpoint despite persistent disturbances. A substantial body of work has developed control-theoretic CRN designs that achieve RPA or near-RPA (*15–17, 30, 72–77*), demonstrated experimental implementations (*9, 14, 19, 20, 22, 78–80*), and provided increasingly general characterizations of RPA-capable network classes (*9, 81–84*). However, even with these characterizations, the remaining design space can still be large, and satisfying structural conditions alone does not guarantee favorable transient performance.

In the following examples, we use GenAI-Net to design molecular controllers that achieve RPA with good dynamic performance (Fig. 4). We formulate RPA as an input–output control task in which an external input u_1 specifies the desired setpoint and a second input u_2 acts as a persistent disturbance by modulating the effective degradation of the regulated species (Fig. 4A,B, left). With a minimal starter I/O CRN containing the regulated output species and controller species, GenAI-Net appends a user-defined number of additional reactions from the library and optimizes a trajectory-level loss (see Methods) that penalizes setpoint-tracking error over time, averaged across multiple setpoints and disturbance strengths. This objective explicitly rewards both fast and smooth dynamics and accurate steady-state regulation across conditions.

Single-process-species yield diverse RPA solutions. In Fig. 4A, the template contains one process species $\mathbf{X_1}$ (regulated output) and two controller species $\mathbf{Z_1}$, $\mathbf{Z_2}$. Allowing up to five appended reactions, GenAI-Net generates a large solution set: the top 100 topologically unique I/O CRNs (gray) robustly converge to the correct setpoints (dashed lines) across scanned (u_1, u_2) conditions, with three representative networks (I/O CRNs 1, 19, and 28) highlighting distinct topologies that realize the same macroscopic behavior. The diversity graph and reactant–product incidence map summarize structural relationships among solutions and recurrent reaction usage in low-loss designs. Notably, the frequently selected sequestration reaction $\mathbf{Z_1} + \mathbf{Z_2} \longrightarrow \varnothing$ effectively rediscovers the antithetic integral feedback (AIF) motif (*15*). At the same time, GenAI-Net repeatedly selects alternative sensing and actuation mechanisms that improve transients, including a conversion-based sensing reaction $\mathbf{X_1} \longrightarrow \mathbf{Z_2}$ (rather than the more common catalytic sensing of $\mathbf{X_1}$) and modified "sequestration" variants in which $\mathbf{Z_1}$ and $\mathbf{Z_2}$ jointly contribute to actuation of $\mathbf{X_1}$ instead of purely annihilating. Together, these results illustrate how GenAI-Net can simultaneously satisfy stringent steady-state objectives (RPA) while discovering reaction-level modifications that yield favorable dynamics.

A minimal single-controller starter I/O CRN reveals compact motifs. Fig. 4B reduces the starter I/O CRN to a single controller species $\mathbf{Z}$, yielding a smaller but still nontrivial solution set (9 topologically unique I/O CRNs). Despite this minimal starting point, GenAI-Net still discovers networks that robustly adapt across setpoints and disturbances, with representative solutions shown (I/O CRNs 2, 4, and 9). Notably, one topology (I/O CRN 4) realizes a particularly compact feedback motif: a *single molecular conversion reaction* simultaneously (i) senses the regulated output and (ii) drives the compensatory action, collapsing sensing and actuation into one reaction channel. This motif is closely related in function to autocatalytic integral control (*16, 72*), but differs in its *non-modular* implementation: rather than a "sense–compute–actuate" separation, the feedback computation is inseparable from the actuation biochemistry itself. The schematic in Fig. 4B (right) illustrates this combined conversion-based mechanism, together with one possible protease-mediated genetic realization in which a protease and its inhibitor form a complex whose conversion back to active protease implements integral-like accumulation while maintaining robustness to persistent disturbances.

Scaling to higher-dimensional regulation. Finally, Supplementary Fig. S1 demonstrates that GenAI-Net scales to a multi-species setting with two process species $\mathbf{X_1}$, $\mathbf{X_2}$ (with $\mathbf{X_2}$ as the regulated output) and four controller species ($\mathbf{Z_1}, \ldots, \mathbf{Z_4}$), where multiple disturbances modulate degradation of each process species. Starting from this higher-dimensional starter I/O CRN and allowing eight additional appended reactions, GenAI-Net again generates a large set of topologically distinct solutions (top 100 unique I/O CRNs) that achieve robust setpoint tracking for the regulated output across the scanned disturbance conditions, with representative topologies shown alongside the aggregate diversity summaries.

Together, these RPA examples show how GenAI-Net can (i) generate *families* of mechanistically distinct adaptive and high performance controllers from a common starter I/O CRN, (ii) recover compact control motifs from minimal starting points, and (iii) scale the same design principles to higher-dimensional regulation problems. We also successfully tested other adaptation tasks, namely habituation and sensitization (*85, 86*) where inputs are allowed to be time varying. The results are reported in the Methods section and in Supplementary Fig. S5.

GenAI-Net generates RPA networks with reduced coefficient of variation in the stochastic reaction kinetics

Intracellular reaction systems can exhibit strongly stochastic dynamics when molecular copy numbers are low (*87*), in contrast to the deterministic regimes considered in our previous examples. Such intrinsic noise can be fundamentally difficult to suppress (*88*). To

evaluate GenAI-Net under these conditions, we tasked the method with constructing RPA networks with tunable means and reduced *Coefficients of Variation* (CV) over time, a property often desirable experimentally as it mitigates dynamical intrinsic noise that manifests as cell-to-cell variability. Similar to the RPA task in the deterministic setting, we selected a starter I/O CRN with one output species, three controller species, and two starting reactions regulated by two inputs representing the setpoint and disturbance, respectively (Fig. 5 top-left). Allowing up to six appended reactions, GenAI-Net discovers more than 40 topologically distinct controllers that achieve stochastic RPA: the mean trajectory tracks the setpoint across disturbance levels u_2 (Fig. 5, top middle). At the same time, the corresponding coefficients of variation (CVs) remain consistently below the Poisson limit $1/\sqrt{\text{mean}}$, i.e., below the open-loop noise level for $\mathbf{X_1}$ in the absence of controller species. This demonstrates that GenAI-Net can jointly satisfy stringent steady-state regulation (RPA), maintain favorable transient performance, and attenuate intrinsic noise under disturbances.

The resulting high-performing networks exhibit recurring motifs reminiscent of antithetic integral feedback (*15*), yet go beyond it in a direction that explicitly targets variance reduction. Whereas classical antithetic integral feedback is known to increase noise relative to open loop (*15*), prior work has sought to mitigate this through added circuitry (e.g., molecular PID) (*23, 89*), modified actuation (*31, 32*), or auxiliary antithetic controllers that regulate higher moments (*90*). In contrast, GenAI-Net automatically uncovers motifs that simultaneously address all three objectives—RPA, fast dynamics, and noise suppression. As in other tasks, we cluster the top-performing solutions by topological similarity (Fig. 5, bottom middle) and report representative circuits (Fig. 5, top right).

Mechanistically, these circuits repeatedly employ conversion-based sensing $\mathbf{X_1} \longrightarrow \mathbf{Z_2}$ (rather than the more common catalytic sensing), and in some families replace pure annihilation $\mathbf{Z_1} + \mathbf{Z_2} \longrightarrow \varnothing$ with a coupled comparison–actuation step $\mathbf{Z_1} + \mathbf{Z_2} \longrightarrow \mathbf{X_1}$, which both removes controller species and directly actuates $\mathbf{X_1}$. In addition, the networks consistently incorporate negative autoregulation on $\mathbf{X_1}$ (Fig. 5, top right and bottom right), providing proportional negative feedback that suppresses stochastic fluctuations. Other solutions add parallel sensing routes—for example, an intermediate controller species that catalytically senses the output while producing both controller species—yielding alternative implementations that preserve adaptation while improving transient and noise properties.

GenAI-Net generates circuits for fate decision

GenAI-Net can also generate biochemical circuits that implement initial-condition-dependent fate decisions, a key mechanism in cell differentiation and tissue development (*91*). Substantial effort has been devoted to the theoretical design and experimental implementation of biomolecular circuits that perform decision making through a range of dynamical mechanisms, including multistable behavior (*92–97*). To illustrate this capability, we tasked GenAI-Net with designing a system of two species and a starter I/O CRN comprised of two reactions (Fig. 6, top left). The resulting network is required to converge to one of two fixed points depending on the initial conditions: if the first species initially dominates, the system converges to $(1, 0)$; otherwise, it converges to $(0, 1)$. Using this setup and a loss function (see Methods) that evaluates performance from several selected initial conditions (Fig. 6 middle left), GenAI-Net can generate more than 40 topologically unique, high-performing networks whose dynamics satisfy the above requirement (Fig. 6). In addition, these networks respond rapidly: for most initial conditions, trajectories converge to the vicinity of the desired fixed points with less than 10 time units. Note that the associated topology map and the reactant–product incidence map are give in Supplementary Fig. S2A).

The generated networks consistently favor certain reactions and exhibit specific patterns, as indicated by the reactant-product incidence map and representative circuits in Fig. 6, demonstrating underlying design principles for this task. As in the previous tasks, the representative networks are chosen from across the clusters of high-performing candidates, where clusters are defined by topological similarity (see Fig. 6 bottom left). These representative networks can be viewed as two independent branches—one for each species—combined via a sequestration reaction $\mathbf{X_1} + \mathbf{X_2} \longrightarrow \varnothing$. This reaction has a relatively high rate constant and acts as a competing module, driving the non-dominant species to approximately zero at steady state and thereby enabling the classification functionality. Moreover, in each branch, an autoregulatory mechanism is provided to regulate the steady-state value of its associated species when it is dominant.

After generation, we perform a post-design generalization analysis for the classifier circuits. Specifically, in addition to the 32 labeled initial conditions used in the design objective, we simulate each 40 top-ranked I/O CRN on an expanded set of 90 initial conditions, some of which closer to the target decision boundary (Fig. 6, bottom). We then recompute the classification loss on this expanded set and compare it to the objective loss, providing a direct check that the learned basin geometry extends beyond the discrete initial conditions used during the design process. This postprocessing step does not affect the design loop; it is used only to assess how well the generated fate-decision mechanisms interpolate to nearby initial states.

Logic circuits

Logic circuits provide a compact and compositional language for programming biochemical systems: by mapping molecular inputs to discrete decisions, they enable modular information processing and serve as reusable building blocks for constructing more complex computations (e.g., multi-layer decision cascades, context-dependent responses, and higher-order CRN "programs" that orchestrate dynamics across many species) (*38, 39, 42*).

With a starter I/O CRN in which four external inputs $u_1, \cdots, u_4$ take Boolean values (low/high; $0/1$) and modulate the corresponding input species $\mathbf{X_1}, \cdots, \mathbf{X_4}$ to regulate an output species $\mathbf{X_5}$, GenAI-Net discovers molecular circuits that implement the irreducible 4-input Boolean function $(u_1 \wedge u_2) \vee (u_2 \wedge u_3) \vee (u_3 \wedge u_4)$ (Fig. 7). Allowing six appended doubly bimolecular mass-action reactions yields

29 top-performing I/O CRNs whose output trajectories cleanly separate into low and high digital levels. We highlight the transient dynamics of three representative solutions (I/O CRNs 1, 2, and 4; colored curves) and mark the low/high setpoints with horizontal dashed lines (Fig. 7).

These solutions share two recurring design features. First, the output is stabilized by an *autocatalytic degradation* motif, which suppresses overshoot and accelerates convergence. Second, the logic is computed in an *analog* manner through direct or catalyzed conversion channels in which pairs of input species promote production of $\mathbf{X_5}$, encoding the conjunctive clauses of the target function (Fig. 7, example topologies). Tight binarization across all 16 input combinations is achieved by adding *catalytic degradation* and *interconversion* reactions among $\mathbf{X_1},\cdots,\mathbf{X_4}$ that tune the effective high-state setpoint, so true cases saturate near unity rather than increasing with input magnitude (Fig. 7), truth tables for I/O CRNs 1, 2, and 4). Note that the associated topology map and the reactant–product incidence map are give in Supplementary Fig. S2B).

Restricting the search to at most five appended reactions yields 40 top-ranked candidates, and we show representative topologies (I/O CRNs 1, 6, and 26; Supplementary Fig. S2C). With this reduced reaction budget, the setpoint-conditioning mechanisms above are typically absent, leading to a systematic overshoot above the nominal high level in the all-inputs-high condition (Supplementary Fig. S2C, time courses). This overshoot does not affect threshold-based classification, but it reveals a trade-off between circuit compactness and tight digital saturation that is alleviated by the sixth reaction in Fig. 7.

Finally, to demonstrate GenAI-Net 's ability to operate on reaction libraries with multi-parameter kinetics, we apply it to a Hill-type production library and successfully generate diverse I/O CRNs, as shown in Supplementary Fig. S3.

GenAI-Net provides biochemical oscillators with specific centers and tunable frequencies

Biological rhythms are widespread in biology, precisely regulating life processes in response to oscillatory environmental cues. Despite the discovery of natural biochemical oscillators (*98–102*), and the development of synthetic ones (*6, 103–105*), designing circuits with specified or tunable oscillatory parameters (frequency, center, amplitude, etc.) remains prohibitively challenging, with only a few rational designs available (*106, 107*). We therefore tasked GenAI-Net with constructing such biochemical oscillators from a starter I/O CRN comprising three species and four reactions where all three species are diluted (top-left panels in Figs. 8 and 9).

We begin with oscillators whose trajectories are required to oscillate around a prescribed center (Fig. 8). In this setting the starter I/O CRN has no inputs, and the objective targets the mean (center) of the output oscillation. GenAI-Net discovers more than 100 topologically distinct high-performing networks, with oscillator centers tightly clustered around the target value (Fig. 8, histogram). Three representative solutions are highlighted with colored trajectories alongside their corresponding topologies, while the topological diversity graph summarizes the breadth of distinct solutions found by GenAI-Net. Interestingly, the sequestration reaction $\mathbf{X_1} + \mathbf{X_3} \longrightarrow \varnothing$ emerges repeatedly in the generated CRNs (see the table in Fig. 8), suggesting that sequestration constitutes a structurally favored mechanism for oscillatory regulation. Notably, related sequestration dynamics play a central role in the mmammalian circadian clock (*108, 109*). We are grateful to Prof. Jae Kyoung Kim for drawing our attention to this connection.

We next consider oscillators with *input-tunable* frequency (Fig. 9). Here the starter I/O CRN includes an input u_1 that modulates the production rate of $\mathbf{X_1}$, providing a tuning knob for the oscillation frequency of the output species $\mathbf{X_3}$. We evaluate three target periods ($T = 10, 15, 20$) and so the input u_1 takes 3 values $u_1 \in \{1/20, 1/15, 1/10\}$, and find that GenAI-Net again produces more than 60 high-performance networks (Fig. 9). Fourier spectra (Fig. 9, right) confirm that, for each input level, the fundamental frequency aligns with the prescribed target period. As in the previous task, the diversity graph and reactant–product incidence map reveal substantial topological variation and recurring reaction usage among the generated solutions (see Supplementary Fig. S4), highlighting the framework's ability to generate multiple distinct implementations of the same dynamical specification.

After generation, we perform a post-design generalization test for the frequency-tunable oscillators. Specifically, although the design objective in Fig. 9 evaluates only a sparse set of target frequencies, we take the top I/O CRNs and simulate them on a dense sweep of 100 input frequencies spanning the same range. To improve frequency estimation, we use a longer simulation horizon ($t_f = 500$), which yields higher-precision frequency measurements and correspondingly lower relative error, including for input frequencies not encountered during design. The resulting input–output frequency curves closely track the identity relation, indicating that many generated oscillators interpolate smoothly across the design range rather than merely matching the discrete targets.

Discussion

We presented GenAI-Net, a generative AI framework for the automated design of chemical reaction networks from high-level dynamical specifications. GenAI-Net exploits the asymmetry between the ease of *evaluating* a proposed CRN by simulation and the difficulty of *discovering* one that satisfies a target behavior. By placing an AI agent in the loop, GenAI-Net iteratively proposes reactions (including kinetic parameters and, when relevant, input influences), evaluates completed networks with a task-defined loss, and refines its proposal distribution to generate diverse, high-performing solutions.

Across a broad set of benchmarks—including dose–response shaping (Hill-type, Michaelis–Menten, ultrasensitive, and non-monotonic profiles), logic circuits implementing complex Boolean functions, classification by initial conditions, oscillators with constrained mean or input-tunable frequency, robust perfect adaptation under deterministic and stochastic dynamics (including simultaneous noise reduction), and habituation/sensitization—GenAI-Net consistently produced families of topologically distinct networks

that match the desired behaviors. Beyond performance, the resulting solution sets enabled mechanistic insight: recurring reaction patterns and cluster-level structure revealed alternative motifs for achieving the same specification.

Despite all these successes, several directions remain to be explored in future work to further improve the method and extend its applicability to more advanced biological studies. From a methodological perspective, the computational efficiency of our generative approach could potentially be improved by adopting more advanced neural network architectures and generative paradigms, such as transformer-based models (*110*), GFlowNets (*111*), and diffusion models (*112*). In addition, incorporating more advanced training strategies—such as Proximal Policy Optimization (PPO) (*113*)—may further accelerate learning and yield more robust performance. Moreover, an adaptive hyperparameter-tuning scheme would help streamline practical use and reduce the burden of manual calibration for end users.

Our current implementation is designed to run on a single computer node (workstation or cluster job); however, because candidate CRNs and evaluation conditions are largely independent, the workflow is inherently parallel. Extending GenAI-Net to distributed, multi-node execution is therefore a natural next step, but it would require careful systems engineering (e.g., asynchronous scheduling of simulations, efficient aggregation of rewards/gradients) to correctly handle the distributed training of our agent.

On the other hand, GenAI-Net naturally extends to richer design settings. One immediate direction is to treat *input assignment* itself as part of the search, allowing the agent to decide not only which reactions to add but also which external signals modulate each propensity. Although we focused here on reaction libraries with mass-action kinetics and hill kinetics (see Supplementary Fig. S3), GenAI-Net is not limited to these settings. The same framework can operate with arbitrary, user-defined reaction libraries in which each reaction carries multiple kinetic parameters and follows more complex propensity forms. The framework can also leverage more sophisticated simulation environments that incorporate additional biological context—such as growth, dilution, resource limitations, burden, spatial effects, or host–circuit interactions—so that designs are optimized under more realistic operating conditions. Beyond the benchmarks explored here, the same loop could be applied to a wider range of objectives and larger networks, including multi-module behaviors and multi-objective specifications. Finally, a complementary avenue is to decouple structure and parameters more explicitly: the agent could focus primarily on proposing reaction *topologies*, while an embedded optimizer (or inner-loop parameter-fitting routine) searches for the best kinetic parameters for each proposed topology, potentially improving sample efficiency and sharpening the topology-level search.

GenAI-Net is intended as a general design engine rather than a solver for a single task class. Users can specify a starter I/O CRN, a reaction library, kinetic semantics, and an evaluation objective, allowing the same framework to be applied across modeling assumptions and application domains. Looking forward, GenAI-Net opens a path toward more programmable molecular engineering, where circuit discovery is guided directly by behavioral specifications and where interpretable reaction-level motifs can be rapidly surfaced, compared, and translated into experimental implementations.

Material & Methods

Components of the RL Loop

We begin by defining the agent and its action space (reaction library), and then proceed to the environment and its state space (I/O CRNs). We then describe the translator that maps CRNs into an agent-interpretable representation, followed by the policy architecture used by the agent.

The agent and action space. The agent is responsible for proposing the next reaction to append to the current incomplete I/O CRN, iterating until the I/O CRN is complete. We formalize the agent's decision as sampling an action from a reaction library, conditioned on the current translated CRN representation. Equivalently, the agent defines a conditional distribution over the next reaction given the current CRN state, i.e., $\mathbb{P}(\text{next reaction} \mid \text{CRN})$, as schematized in Fig. 2A.

Informally, the action space $\mathcal{A}$ corresponds to the set of reactions that may appear in a valid CRN. Because different application settings permit different classes of reactions, we treat $\mathcal{A}$ as a user- or task-chosen *reaction library*, which is a core component of our formulation containing all permissible reaction templates. Let reaction templates in the library be indexed by $\ell \in \{1, \ldots, M\}$, where each template ℓ corresponds to a human-interpretable reaction template (stoichiometry together with associated kinetic parameter variables). When external inputs are present, each reaction may additionally be configured with an *input-influence pattern* indicating which input channels modulate its propensity. Concretely, let there be n_u input channels and let $\mathcal{G}$ denote the (finite) set of admissible input-influence patterns (e.g., $\mathcal{G} \subseteq \{0,1\}^{n_u}$, where a binary vector $g \in \mathcal{G}$ specifies which inputs act on the reaction). Reaction template ℓ has $p_\ell \in \mathbb{N}_0$ parameters with domains $\mathbb{S}_{\ell,\ell'}$ for $\ell' = 1, \ldots, p_\ell$. An action is then a tuple consisting of (i) a discrete reaction-type index, (ii) its continuous parameter vector, and (iii) an input-influence choice:

$$\begin{aligned} \mathcal{A} \;\triangleq\; \{(\ell, \theta, g) \mid\; & \ell \in \{1, \ldots, M\} \\ & \theta = (\theta_1, \ldots, \theta_{p_\ell}),\; \theta_{\ell'} \in \mathbb{S}_{\ell,\ell'}\; \forall \ell',\; g \in \mathcal{G}\;\}\,. \end{aligned} \tag{1}$$

When a task requires no external input assignments, we take $n_u = 0$ and $\mathcal{G} = \{\varnothing\}$, reducing to the (ℓ, θ) action form. Note that $\mathcal{A}$ is a hybrid space with both discrete (reaction template index) and continuous (kinetic parameters) components.

The environment and state space. Let s_t denote the environment state at construction step t, defined as the current (possibly incomplete) I/O CRN obtained by starting from the user-provided starter I/O CRN and appending t reactions. Thus, s_0 is exactly the starter I/O CRN, and for $t > 0$ the state s_t encodes the set of selected reaction templates together with their chosen parameter values and input-influence assignments.

Formally, the state space is the combinatorial set of all such CRNs reachable by appending up to m reactions from the library:

$$\mathcal{S} \triangleq \bigcup_{t=0}^{m-1} \mathcal{S}_t, \quad \text{with } \mathcal{S}_t \triangleq \binom{\mathcal{A}}{t} \triangleq \{\, \mathcal{B} \subseteq \mathcal{A} \mid |\mathcal{B}| = t \,\}. \tag{2}$$

We detail the actual size of the search space under mass action kinetics in a subsequent section. In practice, we enforce validity constraints during construction (e.g., preventing duplicate reaction types and considering the starter I/O CRN). We next describe how a state s_t is mapped into the agent-interpretable representation on which the policy conditions.

The translator. The translator maps a human-readable CRN state s_t (a reaction list with parameters and input influences) into a fixed-format, agent-interpretable observation. Concretely, it encodes, for each library reaction template, whether it is present in the current I/O CRN and, if so, its current parameter value(s) and input-influence assignment. This yields a structured representation of constant dimension across all t, enabling the policy to operate on incomplete CRNs of varying sizes. A running example is shown in Fig. 2A (left), where a toy I/O CRN is translated into this fixed-format encoding using an example library; the reaction highlighted by the red box illustrates how a single library entry is represented.

Policy implementation. For compactness, we describe the policy architecture for the case where input assignment is not required (Fig. 2B). The agent is parameterized by neural-network weights ϕ and defines a conditional distribution over the next reaction type and its kinetic parameters given the current translated CRN state s, where we drop the subscript t for notational convenience. As illustrated in the left panel of Fig. 2B and zoomed-in on the right, the policy is implemented with a shared network trunk that produces an embedding of s, followed by two heads, and is written autoregressively as

$$\pi_\phi(\ell, \theta \mid s) \;=\; \pi_\phi(\ell \mid s)\, \pi_\phi(\theta \mid s, \ell), \tag{3}$$

where we recall that ℓ denotes the reaction template index (or ID) in the library, and θ denotes its parameter values. The *structure head* outputs logits over reaction templates $\ell \in \{1, \ldots, M\}$; logits for reaction templates already present in the current partial CRN are masked to enforce uniqueness, yielding the categorical distribution $\pi_\phi(\ell \mid s_t)$. Conditioned on the sampled template ℓ, the *parameter head* outputs a continuous distribution over θ (we use a lognormal family), yielding $\pi_\phi(\theta \mid s, \ell)$. During sampling, the agent draws $\ell \sim \pi_\phi(\ell \mid s)$ and then $\theta \sim \pi_\phi(\theta \mid s, \ell)$. The corresponding log-probabilities are summed up in log-space. Furthermore, we compute an weighted entropy regularizer by combining the Shannon entropy of the discrete component and the differential entropy of the continuous component:

$$\mathcal{H}^w_{\pi_\phi}(s) \;=\; \mathcal{H}\big[\pi_\phi(\cdot \mid s)\big] \;+\; w\, \mathbb{E}_{\ell \sim \pi_\phi(\cdot \mid s)}\Big[\, \mathcal{H}\big[\pi_\phi(\cdot \mid s, \ell)\big] \,\Big], \tag{4}$$

where the expectation appears because the parameter distribution $\pi_\phi(\theta \mid s, \ell)$ (and thus its differential entropy) depends on the reaction template ℓ, so we average over ℓ under $\pi_\phi(\ell \mid s)$ to obtain a single entropy value for the joint policy at state s. Finally, w is a training hyperparameter controlling the relative strength of the continuous-entropy term, which we discuss in the subsequent training section.

The Training

Training proceeds via episodic construction rollouts of fixed length m. Starting from the initial state s_0 (the starter I/O CRN), the agent repeatedly samples an action and the environment deterministically applies it through the state transition operator $\mathcal{T}$ (by appending the proposed reaction):

$$a_t \sim \pi_\phi(\cdot \mid s_t), \qquad s_{t+1} = \mathcal{T}(s_t, a_t), \qquad t = 0, 1, \ldots, m-1, \tag{5}$$

yielding a trajectory $\tau = (s_0, a_0, \ldots, s_{m-1}, a_{m-1}, s_m)$, where s_m is a *complete* I/O CRN. The completed CRN s_m is then simulated and evaluated using a task-specific objective to yield a scalar loss $\mathcal{L}_{\text{task}}(s_m)$ (defined by the user for each application, as illustrated in our examples). Because partial CRNs can exhibit behavior that is not predictive of the final circuit, we assign reward only at termination and optimize *terminal* performance.

Policy optimization. We define the terminal reward as the negative task loss,

$$\mathcal{R}(\tau) \;\triangleq\; \mathcal{R}(s_m) \;\triangleq\; -\mathcal{L}_{\text{task}}(s_m). \tag{6}$$

Under the standard REINFORCE formulation (*114*) with terminal rewards, the policy is trained by solving the expected-return maximization problem

$$\max_\phi \; \mathbb{E}_{\tau \sim p_\phi}[\mathcal{R}(\tau)] \;=\; \mathbb{E}_{\tau \sim p_\phi}[-\mathcal{L}_{\text{task}}(s_m)]\,, \tag{7}$$

where $p_\phi(\tau)$ denotes the rollout distribution induced by the policy π_ϕ and the deterministic transition $\mathcal{T}$. In GenAI-Net, we build on this objective and introduce four modifications—(i) a risky (top-K) objective that emphasizes high-performing rollouts (*115*), (ii) baseline subtraction using the worst reward among the top-K samples, (iii) hybrid entropy regularization combining discrete (Shannon) and continuous (differential) entropy terms (*116, 117*), and (iv) a self-imitation learning (SIL) term based on a hall-of-fame replay buffer (*118*) —each described in detail in what follows.

Risk-sensitive (top-K) objective with baseline subtraction. Standard REINFORCE maximizes the expected return, which in batch form averages rewards across sampled rollouts. To reduce this averaging effect and emphasize rare high-performing solutions, we skew the objective toward the *upper tail* of the reward distribution. Let q_α denote the α-quantile of the terminal reward $\mathcal{R}(\tau)$ under $\tau \sim p_\phi$ (i.e., $\mathbb{P}(\mathcal{R}(\tau) \le q_\alpha) = \alpha$). The corresponding upper-tail (top-quantile) objective is

$$J_\alpha(\phi) \triangleq \mathbb{E}[\,\mathcal{R}(\tau) \mid \mathcal{R}(\tau) \ge q_\alpha\,],$$

which is closely related to CVaR-style risk-sensitive optimization and the *risky policy gradients* viewpoint (*115*). In practice, we estimate this objective from a batch of N rollouts $\{\tau^{(i)}\}_{i=1}^N$ with terminal rewards $r_i \triangleq \mathcal{R}(\tau^{(i)})$. Let $r_{(1)} \ge \cdots \ge r_{(N)}$ be the sorted rewards and choose $K = \lceil N(1-p) \rceil$, so that the empirical threshold $r_{(K)}$ estimates q_α. We further apply baseline subtraction with the batch baseline defined as the *worst* reward among the top-K samples,

$$\mathcal{B}_{\text{top}K} \triangleq r_{(K)}.$$

The resulting baseline-shifted top-K objective is

$$\begin{aligned} J_{\text{RL}} &\triangleq \frac{1}{K}\sum_{i=1}^{N} \big(r_i - \mathcal{B}_{\text{top}K}\big)\,\mathbf{1}\big\{r_i \ge \mathcal{B}_{\text{top}K}\big\} \\ &= \frac{1}{K}\sum_{i=1}^{K} \big(r_{(i)} - \mathcal{B}_{\text{top}K}\big), \end{aligned} \tag{8}$$

so that only rollouts in the empirical top quantile contribute, with non-negative shifted returns, while all remaining rollouts are excluded from this objective.

Entropy bonus. To encourage exploration in the hybrid action space, we add an entropy bonus computed along construction steps. Using the per-state hybrid policy entropy $\mathcal{H}^w_{\pi_\phi}(s)$ defined in (4), we form

$$J_H^w \triangleq \mathbb{E}_\tau\left[\sum_{t=0}^{m} \mathcal{H}^w_{\pi_\phi}(s_t)\right], \tag{9}$$

and weight it with a training coefficient $\lambda_H \ge 0$. Note that the gradient of this entropy regularization term is implemented using a pathwise stop-gradient approximation. This omits the score-function cross term yielding a lower-variance (but biased) entropy-gradient estimator.

Self-imitation learning (SIL). To prevent "forgetting" rare but valuable solutions, we maintain a hall-of-fame replay buffer containing the best complete CRNs observed so far. Let S_k denote the set of complete CRNs sampled in episode k, and define the cumulative sample set up to episode k as $S_{\le k} = \bigcup_{\bar{k}=1}^{k} S_{\bar{k}}$. For a hall-of-fame capacity k_{HOF}, we define

$$\text{HOF}_k \triangleq \text{TopK}_{k_{\text{HOF}}}(S_{\le k}; \mathcal{R}), \tag{10}$$

i.e., the set of the k_{HOF} highest-reward complete CRNs observed so far. Within the current batch, let

$$R^*_m \triangleq \max_{s \in S_k} \mathcal{R}(s) \tag{11}$$

be the best reward achieved in the current iteration. We then add a SIL objective that increases the likelihood of sampling hall-of-fame solutions only when they outperform the current batch best:

$$\begin{aligned} J_{\text{SIL}}(\phi) &\triangleq -\,\mathbb{E}_{s\sim\text{HOF}_k}\Big[\,(\mathcal{R}(s) - R^*_t)_+ \,\log \bar{p}_\phi(s)\Big], \\ &\text{with} \quad (x)_+ \triangleq \max(0, x), \end{aligned} \tag{12}$$

where $\bar{p}_\phi(s)$ denotes the probability (under the current policy) of generating the complete CRN s, equivalently the likelihood of the action sequence that constructs s, with per-step log-probabilities summed in log-space as described above.

Final objective. We combine the risk-sensitive RL term, the entropy bonus, and SIL:

$$J(\phi) \;\triangleq\; J_{\mathrm{RL}}(\phi) \;+\; \lambda_H\, J_H^w(\phi) \;+\; \lambda_{\mathrm{SIL}}\, J_{\mathrm{SIL}}(\phi), \tag{13}$$

and optimize the agent by minimizing the corresponding loss $\mathcal{L}(\phi) \triangleq -J(\phi)$. The hyperparameters are $(p, k, w, \lambda_H, \lambda_{\mathrm{SIL}}, k_{\mathrm{HOF}})$.

Task-dependent loss functions

Let $\mathcal{N}$ denote an input–output chemical reaction network. Given an input signal u and initial condition x_0, we denote the (vector-valued) network output trajectory by $y(t; x_0, u)$; we omit x_0 and/or u from the notation when they are fixed by the task. Each task defines a loss functional $\mathcal{L}_{\mathrm{task}}(\mathcal{N})$ that quantifies how well $\mathcal{N}$ matches a desired behavior. Unless stated otherwise, we measure output mismatch with an ℓ_1 error

$$e(t) \;\triangleq\; \|y(t; x_0, u) - r\|_1,$$

where r denotes a desired reference (e.g., a setpoint or target trajectory). When performance must hold across multiple tested conditions (inputs, disturbances, and/or initial conditions), we aggregate by averaging over the corresponding evaluation set.

For time-integrated objectives, we use a weighted integral operator

$$\mathcal{I}[e] \triangleq \int_0^{t_{\mathrm{f}}} w_d(t)\, e(t)\, dt, \tag{14}$$

$$w_d(t) = \begin{cases} 0.25, & 0 \le t < t_{\mathrm{f}}/5, \\ 1, & t_{\mathrm{f}}/5 \le t < 4t_{\mathrm{f}}/5, \\ 2, & 4t_{\mathrm{f}}/5 \le t \le t_{\mathrm{f}}, \end{cases} \tag{15}$$

where t_{f} is the final simulation time and $w_d(t)$ emphasizes late-time accuracy (steady-state performance) while still penalizing transient error. Next, we specify the task-specific losses used in this work.

Dose–response shaping. For this task we fix the initial condition to $x_0 = 0$. For each tested input level $u \in \mathcal{U}$, we define the input-conditional loss

$$\mathcal{L}_{\mathrm{DR}}^u(\mathcal{N}) \;\triangleq\; \mathcal{I}\big[\|y(t; u) - r(u)\|_1\big], \tag{16}$$

where $r(u)$ is the target dose–response function. The overall dose–response loss averages across the evaluated input set:

$$\mathcal{L}_{\mathrm{DR}}(\mathcal{N}) \;\triangleq\; \frac{1}{|\mathcal{U}|} \sum_{u \in \mathcal{U}} \mathcal{L}_{\mathrm{DR}}^u(\mathcal{N}). \tag{17}$$

Robust perfect adaptation. For this task, we evaluate performance across setpoints $u_1 \in \mathcal{U}_1$ and disturbances $u_2 \in \mathcal{U}_2$, with reference $r(u_1)$ independent of u_2. We define the overall RPA loss by averaging the time-integrated tracking error over all tested (u_1, u_2) conditions:

$$\mathcal{L}_{\mathrm{RPA}}(\mathcal{N}) \;\triangleq\; \frac{1}{|\mathcal{U}_1|\,|\mathcal{U}_2|} \sum_{u_1 \in \mathcal{U}_1} \sum_{u_2 \in \mathcal{U}_2} \mathcal{I}\left[\|y(t; u_1, u_2) - r(u_1)\|_1\right]. \tag{18}$$

Fate decision circuits. For this task, we evaluate fate decisions across a set of initial conditions $x_0 \in \mathcal{X}_0$, where each x_0 is assigned a basin-specific reference trajectory $r(x_0)$. We define the overall loss by averaging the time-integrated tracking error over all tested initial conditions:

$$\mathcal{L}_{\mathrm{Fate}}(\mathcal{N}) \;\triangleq\; \frac{1}{|\mathcal{X}_0|} \sum_{x_0 \in \mathcal{X}_0} \mathcal{I}\left[\|y(t; x_0) - r(x_0)\|_1\right]. \tag{19}$$

Logic circuits. For each truth-table condition $c \in C_{\mathrm{LOGIC}}$, we measure terminal-time disagreement at t_{f} by

$$\mathcal{L}_{\mathrm{LOGIC}}^c(\mathcal{N}) \;\triangleq\; \|y(t_{\mathrm{f}}; c) - r(c)\|_1. \tag{20}$$

We then average across all truth-table conditions:

$$\mathcal{L}_{\mathrm{LOGIC}}(\mathcal{N}) \;\triangleq\; \frac{1}{|C_{\mathrm{LOGIC}}|} \sum_{c \in C_{\mathrm{LOGIC}}} \mathcal{L}_{\mathrm{LOGIC}}^c(\mathcal{N}). \tag{21}$$

Oscillators. To promote sustained periodic behavior we maximize a *periodicity index* computed from the normalized autocorrelation of the centered output. For a scalar output $y(t)$, define the time-averaged mean

$$\bar{y} \triangleq \frac{1}{t_f}\int_0^{t_f} y(t)\,dt,$$

the centered signal

$$y_c(t) \triangleq y(t) - \bar{y}, \tag{22}$$

and the normalized autocorrelation (for lags $\tau \geq 0$)

$$R_y(\tau) \triangleq \frac{\int_0^{t_f-\tau} \bar{y}(t)\,\bar{y}(t+\tau)\,dt}{\int_0^{t_f} \bar{y}^2(t)\,dt}. \tag{23}$$

We then define the periodicity index as the height of the first positive-lag local maximum of R_y,

$$\rho_1[y] \triangleq \max_{\tau>0}\Big\{R_y(\tau) \;\Big|\; R_y'(\tau) = 0,\; R_y''(\tau) < 0\Big\}. \tag{24}$$

In addition to periodicity, we constrain the oscillation *center* (time average) to match a prescribed target $\mu^\star \in \mathbb{R}$. We thus define the oscillator loss

$$\mathcal{L}_{\mathrm{Osc}}(\mathcal{N}) \;\triangleq\; \big(1-\rho_1[y]\big) \;+\; \lambda_\mu\,\big|\bar{y}-\mu^\star\big|, \tag{25}$$

where $\lambda_\mu \geq 0$ weights the center-matching term.

Oscillators with frequency tuning. For frequency-tunable oscillators we include terms that (i) match a target frequency and (ii) discourage amplitude damping.

We estimate the oscillation period by averaging successive inter-peak intervals. Let t_i denote successive peak times of $|y_c(t)|$, let n_p be the number of detected peaks in $[0, t_f]$, and define

$$T[y] \triangleq \frac{1}{n_p-1}\sum_{i=1}^{n_p-1}\big(t_{i+1}-t_i\big). \tag{26}$$

To quantify damping, define peak amplitudes $A_i \triangleq |y_c(t_i)|$ and the damping index

$$\zeta[y] \triangleq \frac{1}{n_p-1}\sum_{i=1}^{n_p-1}\frac{A_i}{A_{i+1}}. \tag{27}$$

For a single evaluated condition $u \in \mathcal{U}$ (where u specifies the target frequency), we define the per-condition loss

$$\begin{aligned}\mathcal{L}^u_{\mathrm{OscF}}(\mathcal{N}) \;\triangleq\; \big(1-\rho_1[y(\cdot;u)]\big) \;+\; \lambda_f\left|\frac{1}{T[y(\cdot;u)]}-u\right| \\ +\; \lambda_d\big(\zeta[y(\cdot;u)]-1\big),\end{aligned} \tag{28}$$

where $\rho_1[\cdot]$ is the periodicity index defined above, and $\lambda_f, \lambda_d \geq 0$ are weights. The overall loss averages across the evaluated conditions:

$$\mathcal{L}_{\mathrm{OscF}}(\mathcal{N}) \;\triangleq\; \frac{1}{|\mathcal{U}|}\sum_{u\in\mathcal{U}}\mathcal{L}^u_{\mathrm{OscF}}(\mathcal{N}). \tag{29}$$

Habituation and sensitization losses. In this application, we target two adaptive response classes—habituation and sensitization (*85, 86*)—using a common loss construction based on three qualitative hallmarks: (i) a systematic change in response amplitude across repeated pulses within a pulse train (decreasing for habituation, increasing for sensitization), (ii) recovery of the output toward its pre-stimulus fixed point after a sufficiently long OFF gap (*spontaneous recovery*), and (iii) a stronger cumulative effect under higher-frequency stimulation (*frequency sensitivity*). The two tasks differ only in the direction of the within-train peak progression; all other terms are shared.

We evaluate these hallmarks over input levels $u \in \mathcal{U}$ and pulse shapes $\mathbf{p} \in \mathcal{P}$, where each pulse shape $\mathbf{p} = (t_{\mathrm{on}}, t_{\mathrm{off}})$ defines both a stimulation frequency and a duty cycle. Each condition $(u, \mathbf{p})$ consists of three phases: a pre-gap pulse train, an OFF gap of duration t_{gap}, and a post-gap pulse train (see Supplementary Fig. S5B,C, red pulses). Let $y(t; u, \mathbf{p})$ denote the output trajectory, and let $a_k^{\mathrm{pre}}(u, \mathbf{p})$

and $a_k^{\text{post}}(u, \mathbf{p})$ denote the detected peak amplitudes (measured relative to the pre-stimulus fixed point) during the pre-gap and post-gap pulse trains, respectively.

To treat habituation and sensitization in a unified way, we define the sign

$$s \triangleq \begin{cases} +1, & \text{habituation}, \\ -1, & \text{sensitization}, \end{cases} \tag{30}$$

so that the desired trend is always encoded by

$$s\,(a_{k+1} - a_k) \le 0,$$

that is, decreasing peaks for habituation and increasing peaks for sensitization.

For a fixed condition $(u, \mathbf{p})$, let n_{pre} be the number of detected peaks in the pre-gap pulse train, and define the signed consecutive-peak ratios

$$r_k(u, \mathbf{p}) \triangleq \left(\frac{a_{k+1}^{\text{pre}}(u, \mathbf{p})}{a_k^{\text{pre}}(u, \mathbf{p}) + \varepsilon} \right)^s, \qquad k = 1, \ldots, n_{\text{pre}} - 1, \tag{31}$$

where $\varepsilon > 0$ is a small constant for numerical stability. We quantify the within-train trend by the log-median of these ratios:

$$\mathcal{L}_{\text{trend}}^{u,\mathbf{p}}(\mathcal{N}) \triangleq \log\Big(\text{median}\{r_k(u, \mathbf{p})\}_{k=1}^{n_{\text{pre}}-1} + \varepsilon \Big). \tag{32}$$

This term is small when the pre-gap pulse amplitudes evolve in the desired direction.

To enforce spontaneous recovery, let t_{gap}^- denote the end of the OFF gap and let $y_{\text{fp}}(u)$ be the pre-stimulus fixed-point output under zero input. We define

$$\begin{aligned} \mathcal{L}_{\text{gap}}^{u,\mathbf{p}}(\mathcal{N}) \triangleq\ & \left| y(t_{\text{gap}}^-; u, \mathbf{p}) - y_{\text{fp}}(u) \right| \\ & + \left[a_1^{\text{pre}}(u, \mathbf{p}) - a_1^{\text{post}}(u, \mathbf{p}) \right]_+, \end{aligned}$$

which jointly encourages the output to relax back toward its original fixed point during the gap and to recover a comparable first-pulse response after the gap.

To reward stronger adaptation under higher-frequency stimulation, we define the signed adaptation score

$$\Delta(u, \mathbf{p}) \triangleq -\, s\, \frac{a_{n_{\text{pre}}}^{\text{pre}}(u, \mathbf{p}) - a_1^{\text{pre}}(u, \mathbf{p})}{a_1^{\text{pre}}(u, \mathbf{p}) + \varepsilon}, \tag{33}$$

so that larger $\Delta(u, \mathbf{p})$ always corresponds to a stronger effect, whether habituation (larger decrease) or sensitization (larger increase). Ordering pulse shapes by increasing frequency as $\mathbf{p}_1, \ldots, \mathbf{p}_{|\mathcal{P}|}$, we penalize violations of monotonic strengthening:

$$\mathcal{L}_{\text{freq}}^{u}(\mathcal{N}) \triangleq \frac{1}{|\mathcal{P}| - 1} \sum_{j=1}^{|\mathcal{P}|-1} \left[\Delta(u, \mathbf{p}_j) - \Delta(u, \mathbf{p}_{j+1}) \right]_+, \tag{34}$$

with $\mathcal{L}_{\text{freq}}^{u}(\mathcal{N}) \equiv 0$ when $|\mathcal{P}| = 1$.

The per-condition loss combines the within-train trend and spontaneous-recovery terms:

$$\mathcal{L}_{\text{Hab/Sens}}^{u,\mathbf{p}}(\mathcal{N}) \triangleq \mathcal{L}_{\text{trend}}^{u,\mathbf{p}}(\mathcal{N}) + \lambda_{\text{gap}} \mathcal{L}_{\text{gap}}^{u,\mathbf{p}}(\mathcal{N}), \tag{35}$$

and the overall loss averages across inputs and pulse shapes, with an additional frequency-sensitivity penalty:

$$\begin{aligned} \mathcal{L}_{\text{Hab/Sens}}(\mathcal{N}) \triangleq\ & \frac{1}{|\mathcal{U}|\,|\mathcal{P}|} \sum_{u \in \mathcal{U}} \sum_{\mathbf{p} \in \mathcal{P}} \mathcal{L}_{\text{Hab/Sens}}^{u,\mathbf{p}}(\mathcal{N}) \\ & + \lambda_{\text{freq}} \frac{1}{|\mathcal{U}|} \sum_{u \in \mathcal{U}} \mathcal{L}_{\text{freq}}^{u}(\mathcal{N}), \end{aligned}$$

where $\lambda_{\text{gap}}, \lambda_{\text{freq}} \ge 0$ are weighting coefficients.

Stochastic objectives (mean and noise). For stochastic CRNs we optimize moments of the output. Under an evaluation condition $c \in \mathcal{C}$, let

$$\mu(t; c) \triangleq \mathbb{E}[y(t; c)]$$

denote the output mean. We define the mean-tracking loss

$$\mathcal{L}_{\text{MEAN}}^{c}(\mathcal{N}) \triangleq \mathcal{I}\left[\|\mu(t; c) - r(c)\|_1 \right]. \tag{36}$$

To penalize variability, we use the (component-wise) coefficient of variation

$$\mathrm{CV}(t;c) \triangleq \frac{\sigma(t;c)}{\mu(t;c)+\varepsilon},$$

where $\sigma(t;c)$ is the output standard deviation and $\varepsilon > 0$ avoids division by zero. The variability loss is

$$\mathcal{L}^c_{\mathrm{CV}}(\mathcal{N}) \triangleq \mathcal{I}\left[\|\mathrm{CV}(t;c)\|_1\right]. \tag{37}$$

We combine the two objectives as

$$\mathcal{L}^c_{\mathrm{MEAN+CV}}(\mathcal{N}) \triangleq \mathcal{L}^c_{\mathrm{MEAN}}(\mathcal{N}) + \lambda_{\mathrm{CV}}\mathcal{L}^c_{\mathrm{CV}}(\mathcal{N}), \tag{38}$$

with $\lambda_{\mathrm{CV}} \geq 0$. The overall stochastic loss averages across evaluated conditions:

$$\mathcal{L}_{\mathrm{MEAN+CV}}(\mathcal{N}) \triangleq \frac{1}{|C|}\sum_{c\in C}\mathcal{L}^c_{\mathrm{MEAN+CV}}(\mathcal{N}), \tag{39}$$

(or, when conditions are grouped, by averaging within groups and then across groups).

Size of the search space

Candidate topologies scale combinatorially with the reaction-template library. Let $\mathbb{L}$ be the library of admissible reaction templates and let $\mathbb{L}_0 \subseteq \mathbb{L}$ denote templates fixed a priori (e.g., those in the starter I/O CRN) or excluded from sampling. The effective library is $\mathbb{L}^- = \mathbb{L} \setminus \mathbb{L}_0$. Let $M \triangleq |\mathbb{L}^-|$ be the size of the effective library. Appending m reactions corresponds to selecting a size-m subset of $\mathbb{L}^-$, so the number of topologically distinct candidates is

$$n_{\mathrm{CRN}}(m, M) = \binom{M}{m}. \tag{40}$$

The library size M depends on the reaction family and the number of species.

Mass-action kinetics. Under mass-action kinetics, a template reaction is specified by a reactant and a product complex (including the null complex). We say that the reaction is of doubly order o if the reactant and product complexes contain at most o molecules drawn from a species set of size n. Therefore, given n and o, the number of all possible complexes is

$$n_c(n, o) = \sum_{i=0}^{o} \mathrm{MultiSet}(n, i), \tag{41}$$

where

$$\mathrm{MultiSet}(a, b) \triangleq \binom{a+b-1}{b} \tag{42}$$

denote the number of multisets of cardinality b drawn from a set of size a. It can be shown that the sum can be simplified to yield

$$n_c(n, o) = \binom{n+o}{n}. \tag{43}$$

Finally, pairing distinct reactant/product complexes yields

$$M = n_c(n, o)\left[n_c(n, o) - 1\right] + 1, \tag{44}$$

where the final term accounts for the null reaction (empty-to-empty).

For instance, for $n = 3$ species, order $o = 2$, and $m = 5$ reactions, the library size is $M = 91$, and the number of possible CRN topologies is $n_{\mathrm{CRN}} = 46,504,458$. Of course this excludes the rate parameters which further expands the space. Adding one more reaction, that is $m = 6$, without changing the number of species and reaction order increases the possibilities by over an order of magnitude to $n_{\mathrm{CRN}} = 666,563,898$. Allowing $n = 4$ species increases the library size to $M = 211$ and the number of CRN topologies to $n_{\mathrm{CRN}} = 114,081,819,852$.

Hill-type kinetics. For Hill-type (production-only) templates, the number of distinct product complexes of size up to n_{prod} is

$$\sum_{n_i=1}^{n_{\text{prod}}} \text{MultiSet}(n, n_i).$$

Signed regulator (activator or repressor enzyme) sets of size up to n_{REG} contribute a factor

$$\sum_{n_i=1}^{n_{\text{REG}}} \binom{n}{n_i} 2^{n_i}, \tag{45}$$

so the Hill library size is

$$M = \sum_{n_i=1}^{n_{\text{prod}}} \text{MultiSet}(n, n_i) \sum_{n_i=1}^{n_{\text{REG}}} \binom{n}{n_i} 2^{n_i}. \tag{46}$$

For example, allowing only on product per reaction ($n_{\text{prod}} = 1$) and up to two regulators ($n_{\text{REG}} = 2$), we have $M = 432$ reactions.

Implementation and computational resources

The GenAI-Net implementation will be made fully available on GitHub with the final version of this preprint. The software is written in Python, using PyTorch for the reinforcement-learning loop and PyCUDA for GPU-accelerated stochastic simulation.

Deterministic analyses other than the dose-response experiments were run on a workstation (WS1) equipped with an AMD Ryzen Threadripper PRO 7985WX CPU and an NVIDIA RTX 4500 Ada GPU, with computations distributed across 64 CPU cores. Stochastic simulations, which are predominantly GPU-bound, were performed on a machine (WS2) equipped with an Intel Xeon W-2123 CPU and NVIDIA Titan RTX GPUs; these resources were provided by the D-BSSE ETH Zurich. Dose-response experiments were conducted on a computing cluster (CLS) with Intel Xeon Gold 6330 CPUs using 40 CPU cores and with Nvidia A800 GPUs, provided by the Academy of Mathematics and Systems Science, Chinese Academy of Sciences.

Deterministic simulations for logic-circuit training runs were executed on WS1. For the logic-circuit experiments (5 species, 6 reactions), training with batch size 1024 for 300 epochs required approximately 5 hours. Stochastic simulations were executed on WS2: for all SSA-based evaluations we used 1000 SSA simulations per batch element. The stochastic RPA experiment with Coeficient of Variation (CV) reduction (batch size 160, 600 epochs) required approximately 20 hours.

User workflow

This section provides a high-level guide for applying our method. We outline how to define a problem, which hyperparameters matter in practice, how to configure logging, how to store trained models, and how to visualize results.

Conceptualization of the problem

This is the first—and one of the most critical—steps for a successful application of our method. In our setting, it starts with choosing the application domain: whether to use a deterministic or stochastic CRN interpretation, and selecting a reaction library. These choices determine which backend is used to evaluate candidate CRNs inside the training loop.

To define the task, the user must specify (i) a loss function, (ii) a set of inputs, and (iii) initial conditions. Since many applications can be formulated as transient tracking problems, we provide utilities to construct distance-from-reference loss functions among others.

As an example, consider a task that searches for circuits exhibiting RPA. We used a single initial condition (the zero state), allowed deterministic CRNs with mass-action kinetics, selected a small set of inputs and disturbances ($u_{\text{in}} \in \{1, 2, 3\}$ and $u_{\text{disturbance}} \in \{0.5, 1, 1.5\}$), and defined the loss as the time integral of the tracking error between a designated output species and a reference proportional to u_{in} as detailed in a previous section. This encourages both accurate setpoint tracking and fast responses.

Hyperparameters

After defining the problem, the method hyperparameters must be specified; in practice, only a small subset has a noticeable effect on performance. A compact reference is provided in Table 1, which reports the values used for this task (and can serve as a reasonable default configuration).

Training and logging

We provide an interface for experiment tracking via `Comet.ml`, enabling users to monitor training progress and to log each component of the objective (e.g., reward, entropy bonuses, and auxiliary terms) separately. In addition, we support task-dependent visualizations of candidate CRN behaviors. For example, in the setting considered above, we render the transient trajectories of the designated output species under multiple input levels and disturbance instances to directly assess tracking and robustness.

At the end of training, users can export the trained RL agent together with a snapshot of the Hall of Fame. The Hall of Fame stores the highest-performing CRNs encountered during optimization and should be interpreted as the primary output of the search procedure (i.e., a curated set of promising designs).

Parameters of discovered I/O CRNs

All parameters for the discovered I/O CRNs presented in this manuscript are provided in the Supplementary Material. We also provide an explicit rewriting of each CRN in terms of reaction equations (as a complement to the graphical representations).

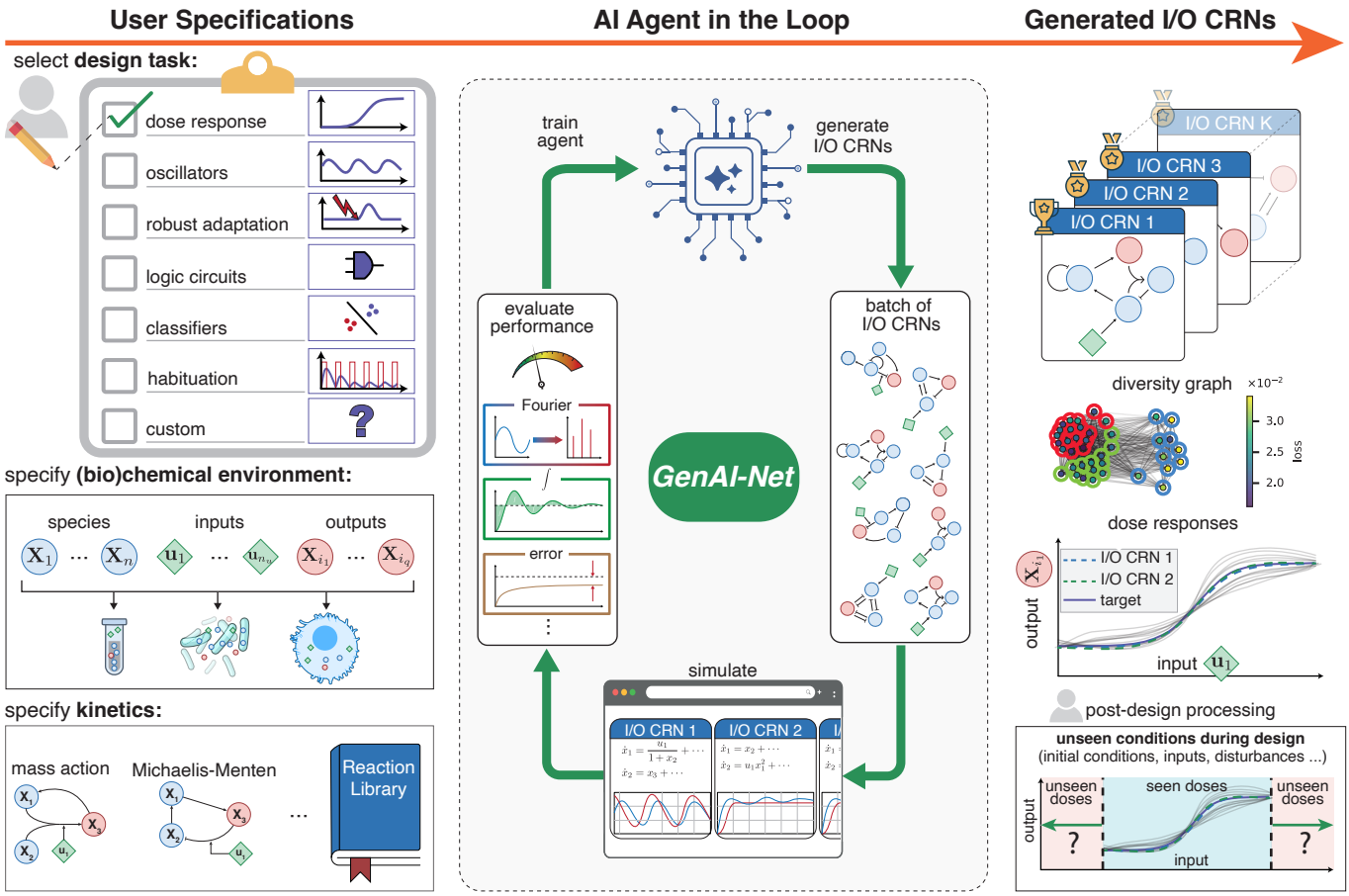


Figure 1: **GenAI-Net overview: a generative AI agent for automatic design of input–output chemical reaction networks (I/O CRNs). User specifications.** Users begin by selecting a desired design *task* (e.g. dose–response, oscillators, robust perfect adaptation, logic circuits, classifiers, habituation, or any custom task). They then specify the intended (bio)chemical operating environment, including the set of chemical species that may appear in the network, the designation of *inputs* (e.g., externally set signals u_1) and *outputs* (regulated/readout species), and any contextual constraints implied by the application setting (e.g. in vitro, cellular contexts, or others). Finally, users choose an appropriate kinetic model class (e.g., mass-action or Michaelis–Menten) and provide (or select) an available *reaction library* containing M permissible reactions from which candidate networks may be assembled. **AI agent in the loop.** Given these specifications, GenAI-Net iteratively *generates* candidate I/O CRNs, *simulates* their dynamics under the chosen kinetics, and *evaluates performance* against the task objective using quantitative metrics (e.g. time-domain error and frequency-domain/Fourier features). These evaluations provide a learning signal used to *train the agent*, closing the design loop: the agent proposes networks conditioned on prior performance, progressively improving the generated candidates. The output of a design run is a *batch of I/O CRNs* sampled from the learned search policy, enabling downstream selection, analysis, and implementation. **Generated I/O CRNs and behaviors.** GenAI-Net returns multiple top-ranked candidate I/O CRNs, each represented by a specific reaction topology, parameters, and its corresponding predicted input–output behavior. Diversity graphs can used to monitor the topological diversity of the generated I/O CRNs. The example to the right highlights how topologically distinct candidate networks can realize similar target specifications (e.g., matching a desired dose–response curve) while differing in internal reaction structure, enabling users to trade off performance, simplicity, and implementability when choosing a final design. The resulting designs can then be evaluated under conditions not used in the design loop. For example, the bottom-right panel illustrates a post-design generalization test in which the generated I/O CRNs are simulated across dose levels that were not included during design.

Hyperparameter	Description	Default value	Impact
Entropy			
λ_H	Weight of the entropy bonus to promote exploration.	5×10^{-3}	high
w	Entropy weight encouraging diversity in proposed CRN parameters.	$1/5$	high
Risk			
p	Risk level: use only the top p-percentile topologies to compute updates.	0.90	high
Self-Imitation Learning (SIL)			
λ_{SIL}	Weight of the SIL loss to reduce forgetting by reinforcing past good solutions.	1.0	low
$n_{\text{SIL-HOF}}$	Replay buffer size (Hall-of-Fame): number of CRNs stored for SIL.	50	low
Agent			
n_{epochs}	Number of training epochs.	300	moderate
net_width	Hidden-layer width of the policy feed-forward network.	10240	low
net_depth	Number of hidden layers in the policy feed-forward network.	5	low
batch_size	mini-batch size	1024	moderate

Table 1: Hyperparameters used for the GenAI-Net.

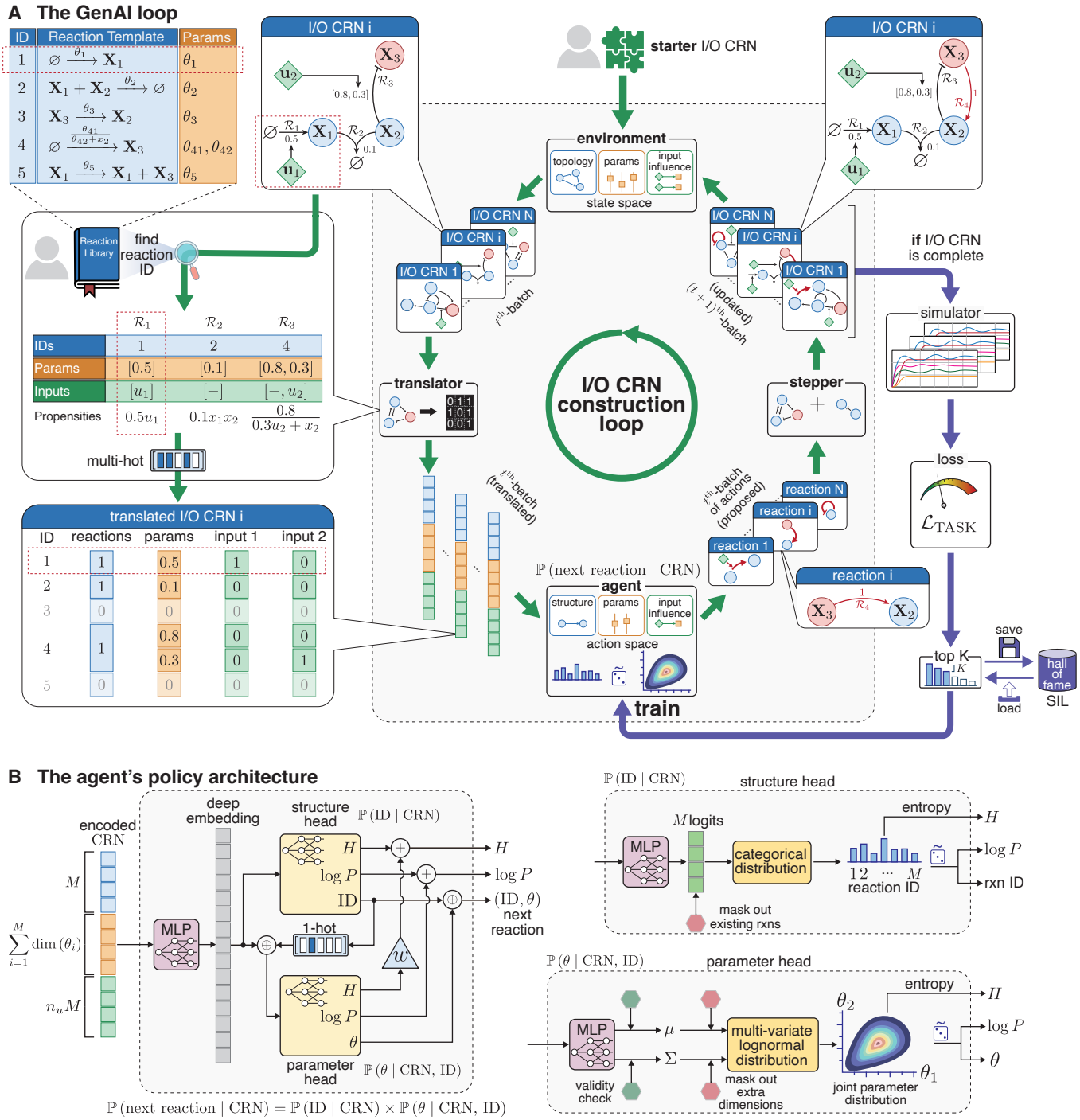


Figure 2: **Method overview: the GenAI-Net generative design loop and policy architecture. (A) The GenAI loop.** Starting from a user-provided starter I/O CRN (top), GenAI-Net grows candidate networks by sequentially selecting reaction templates from a *reaction library* of size M and assigning their kinetic parameters. A candidate I/O CRN (left; "I/O CRN i") is represented as a set of reaction IDs and associated rate parameters (table), together with any designated external inputs that modulate propensities. This representation is translated into an agent-interpretable form (bottom left), where each reaction is encoded as a multi-hot entry indicating whether it is present, its parameter value(s), and which input channels influence its propensity (illustrated as "input 1" and "input 2"). This translation maps the I/O CRN state maintained by the environment into the fixed-format representation consumed by the agent. Conditioned on this translated representation, the *agent* (bottom center) proposes an action (e.g., adding a new reaction and sampling its kinetic parameters). A *stepper* (center right) applies the action to update the I/O CRN in the *environment* (producing the next incomplete I/O CRN; top center), and the *translator* (center left) is applied again to yield the agent's next observation. Once a terminal condition is met (e.g. the required number of reactions to complete the I/O CRN is reached), the environment uses the updated I/O CRN to build the corresponding dynamical model under the chosen kinetic semantics and passes it to the *simulator* (right) and then to the loss module to evaluate the resulting trajectories and compute task-specific loss $\mathcal{L}_{\text{TASK}}$. Across many rollouts, GenAI-Net maintains a "hall of fame" of high-performing solutions, and uses these elite trajectories to refine the training of the agent via self-imitation learning (SIL), biasing future sampling toward successful reaction sequences and parameterizations seen in previous iterations. **(B) The agent's policy architecture.** The policy maps the current incomplete I/O CRN (encoded as the stacked multi-hot reaction/parameter/input tensor) to a *deep embedding*, which feeds two coupled heads. The *structure head* produces logits over the discrete action space of reaction IDs, defining a categorical distribution $\mathbb{P}(\text{ID} \mid \text{CRN})$; already-present reactions are masked to prevent redundant selection, and an entropy term encourages exploration during training. Given the selected reaction ID, the *parameter head* outputs a joint distribution over the continuous kinetic parameters $\mathbb{P}(\boldsymbol{\theta} \mid \text{CRN}, \text{ID})$. Together, these heads define the factored policy $\mathbb{P}(\text{next reaction} \mid \text{CRN}) = \mathbb{P}(\text{ID} \mid \text{CRN})\mathbb{P}(\boldsymbol{\theta} \mid \text{CRN}, \text{ID})$, enabling end-to-end learning of both network topology and kinetics within the closed-loop generation–simulation–training pipeline.

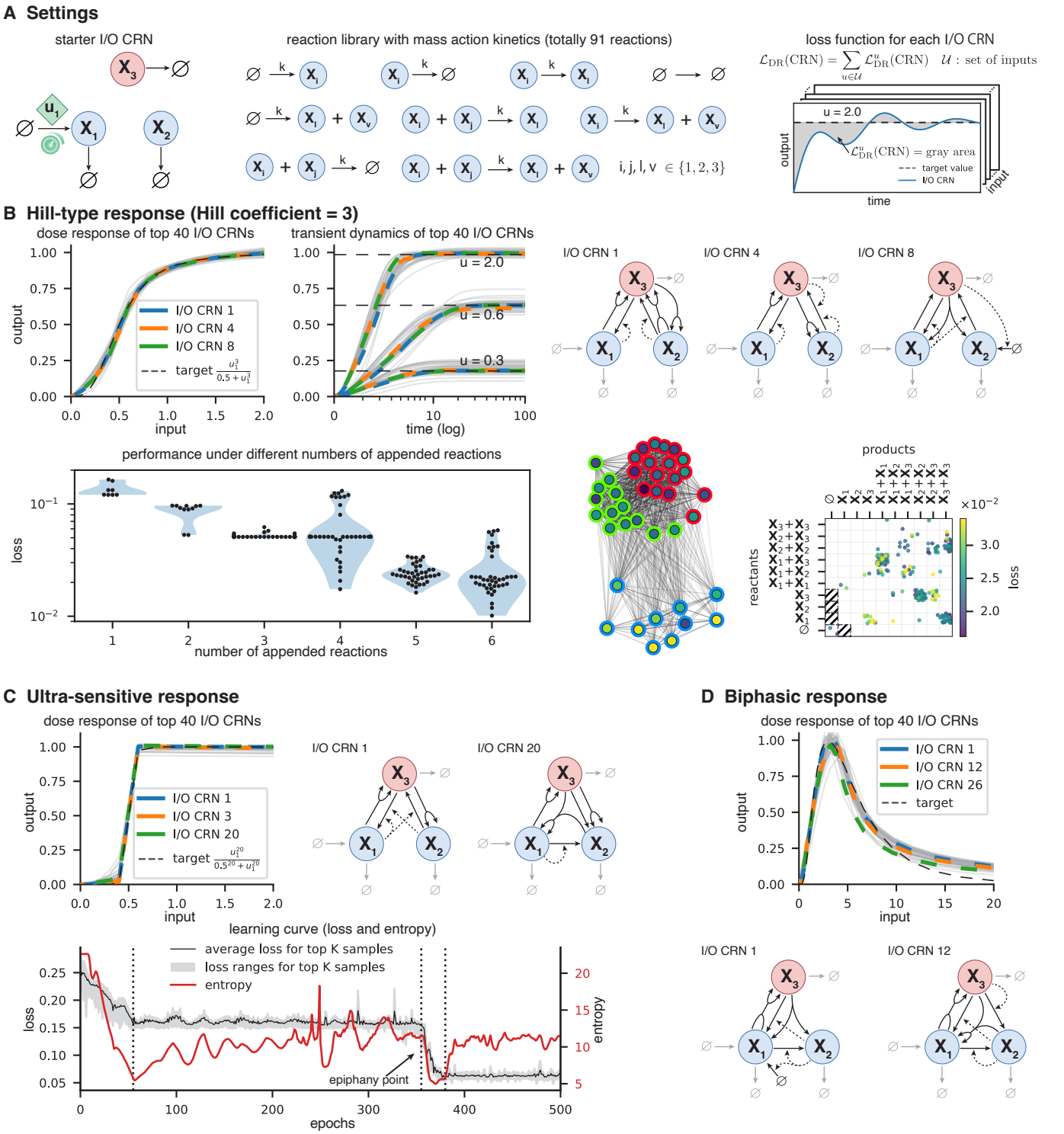


Figure 3: **Dose Response. (a) Problem setting for generating I/O CRNs exhibiting desired dose responses.** GenAI-Net starts from a starter I/O CRN (left) consisting of three species, including an output species $\mathbf{X_3}$, and four reactions. The rate of one of the reactions is modulated by an external input u_1. From this starter I/O CRN, GenAI-Net generates candidate I/O CRNs by appending up to five doubly bimolecular mass-action reactions from a library (middle) and optimizes the average loss of the top 5% generated networks together with an entropy term to boost network diversity. The loss of each network is defined as a weighted L_1 norm of the difference between the system trajectory and the desired response over time, averaged across many different input conditions (see right). **(b) Generation of networks exhibiting Hill-type responses.** GenAI-Net is capable of generating more than 40 topologically unique networks whose steady-state dose-response closely matches the target Hill function (first panel in the first row), while also exhibiting fast and smooth transient dynamics (second panel in the first row). In these plots, the colored lines represent the performance of three relatively topologically distinct networks (top right), while the gray lines indicate the performance of the remaining networks among the top 40. The graph visualization (the middle panel in the second row) summarizes topological diversity across the top 40 generated solutions: nodes denote I/O CRN topologies, edge grayscale encodes pairwise Hamming distance (the number of differing reactions), node fill color indicates loss, and node outline color indicates clusters identified automatically via the Louvain method. The reactant–product incidence map (right bottom) summarizes reaction usage across the generated collection of I/O CRNs: each point corresponds to the reactants and their products, and points are colored by the loss associated with I/O CRNs in which the reaction appears (color bar; scale noted). The bottom-left panel illustrates how the performance of the top I/O CRNs changes with the number of appended reactions. **(c)–(d) Generation of networks exhibiting ultrasensitive and biphasic responses, respectively.** The bottom panel in (c) shows the evolution of the loss and entropy over epochs during the learning procedure.

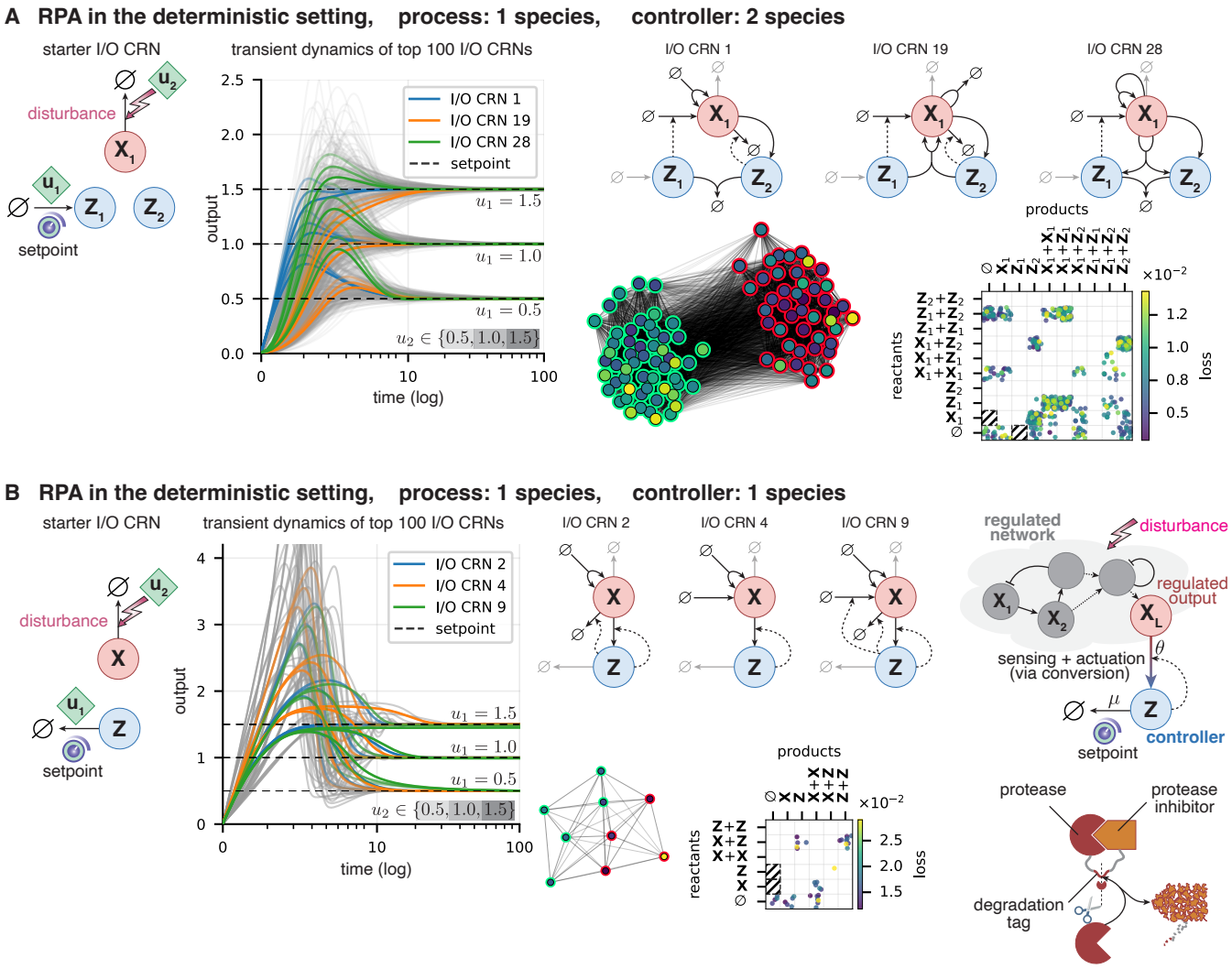


Figure 4: **Robust perfect adaptation: setpoint tracking and disturbance rejection. (a)** GenAI-Net begins with a starter I/O CRN (left) consisting of three species—the regulated output $\mathbf{X_1}$ (red) and controller species $\mathbf{Z_1}$, $\mathbf{Z_2}$ (blue)—and two reactions whose rates are modulated by external inputs u_1 and u_2. The input u_1 tunes the desired setpoint by adjusting the production of $\mathbf{Z_1}$, while u_2 acts as a disturbance by modulating the degradation rate of the output species $\mathbf{X_1}$. From this starter I/O CRN, GenAI-Net generates candidate I/O CRNs by appending up to five doubly bimolecular mass-action reactions, optimizing a loss defined as a weighted L_1 norm of the setpoint-tracking error over time, averaged across three setpoints ($u_1 \in \{0.5, 1.0, 1.5\}$) and three disturbance magnitudes ($u_2 \in \{0.5, 1.0, 1.5\}$); see the loss definition in Methods. The time-course plot shows the regulated output versus log time for the top 100 topologically unique I/O CRNs (thin gray trajectories), with three representative solutions highlighted (colored curves; I/O CRNs 1, 19, and 28). For each highlighted I/O CRN, transparency encodes the disturbance strength set by u_2 (lighter to darker shades). All responses demonstrate robust convergence toward the desired setpoint levels (horizontal dashed lines) across the scanned disturbance and setpoint settings. Example I/O CRN topologies for the highlighted networks are shown to the right. The graph visualization (bottom center) summarizes topological diversity across the 100 generated solutions. The reactant–product incidence map (bottom right) summarizes reaction usage across the generated collection of I/O CRNs. **(b)** A minimal template with a single controller species $\mathbf{Z}$ yields a smaller GenAI-Net-generated *solution set* (9 topologically unique I/O CRNs). Here, up to four reactions are added. As in (a), the center panel shows output trajectories for all generated solutions (gray), with three representative networks highlighted (I/O CRNs 2, 4, and 9) and their corresponding topologies shown to its right. The reactant–product incidence map and the topological diversity graph are also shown. The schematic to the far right highlights a control motif discovered by GenAI-Net (I/O CRN 4) that closely resembles the autocatalytic integral controller reported in (*16, 72*), yet departs from the standard control-theoretic separation between *sensing* and *actuation*. Here, a single molecular conversion reaction performs both operations simultaneously, effectively encapsulating the feedback computation within one reaction channel. The cartoon genetic implementation (bottom far right) illustrates one possible realization via protease-mediated degradation, in which a protease (and its inhibitor) targets a degradation tag to implement the required effective controller reactions.

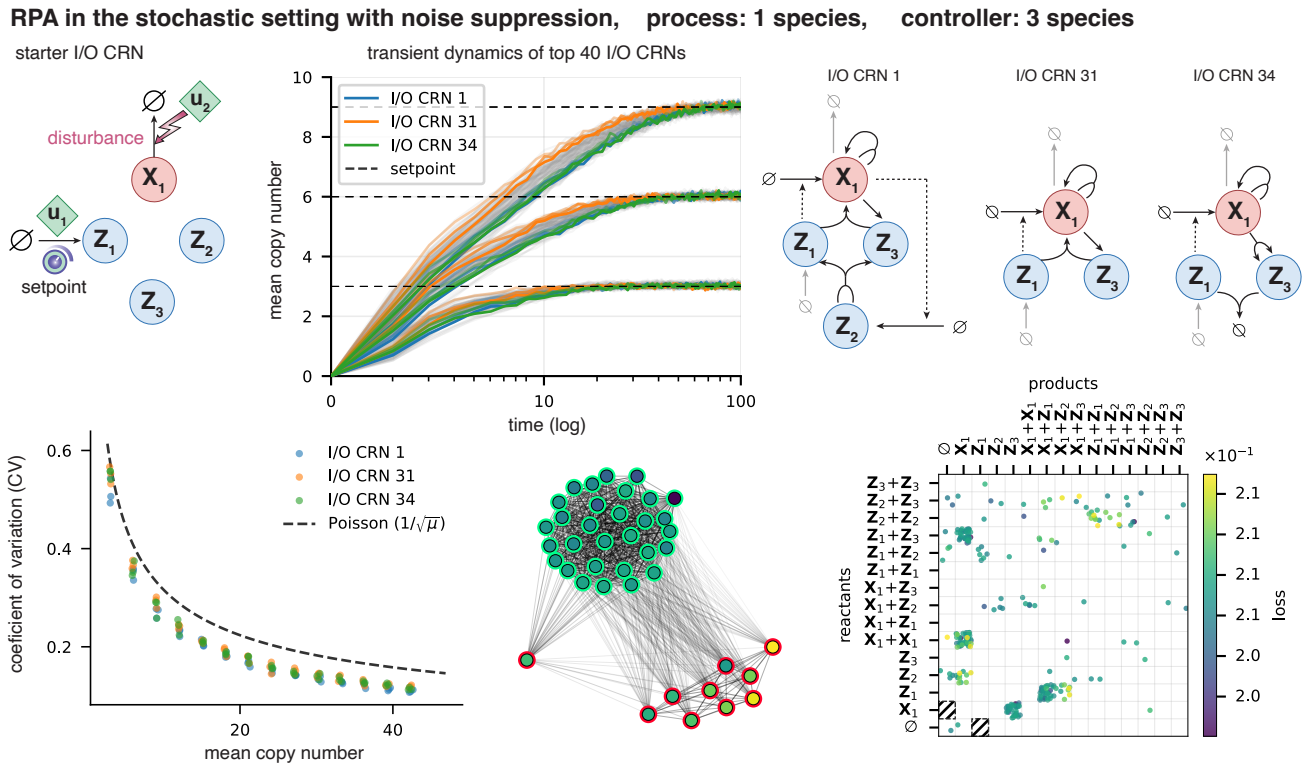


Figure 5: **Stochastic robust perfect adaptation with noise suppression: setpoint tracking with disturbance rejection while minimizing coefficient of variation.** GenAI-Net starts from a stochastic starter I/O CRN (top left) consisting of an output species $\mathbf{X_1}$ (red) and three controller species $\mathbf{Z_1}$–$\mathbf{Z_3}$ (blue). The input u_1 specifies the desired mean setpoint for $\mathbf{X_1}$, while u_2 acts as a disturbance that must be rejected, i.e., changes in u_2 should not alter the steady-state mean of $\mathbf{X_1}$. From this template, GenAI-Net generates candidate I/O CRNs by appending additional reactions and optimizing a stochastic objective that enforces robust perfect adaptation (RPA) in the mean—setpoint tracking with rejection of u_2—while simultaneously minimizing output noise, quantified via the coefficient of variation (CV) of $\mathbf{X_1}$ (see Methods). The time-course panel (top center) shows mean trajectories of $\mathbf{X_1}$ versus log time for the generated *solution set* (thin gray trajectories), with three representative solutions highlighted (colored curves; I/O CRNs 1, 31, and 34) and target setpoint levels indicated by horizontal dashed lines. Across scanned conditions, highlighted solutions robustly converge to the prescribed mean setpoints while maintaining adaptation despite the disturbance u_2. Noise performance is summarized in the CV panel (second row, left): for the three representative networks, the CV remains below the Poissonian reference level (black curve), indicating sub-Poisson fluctuations and improved noise suppression relative to the open-loop configuration where no controller species are present.

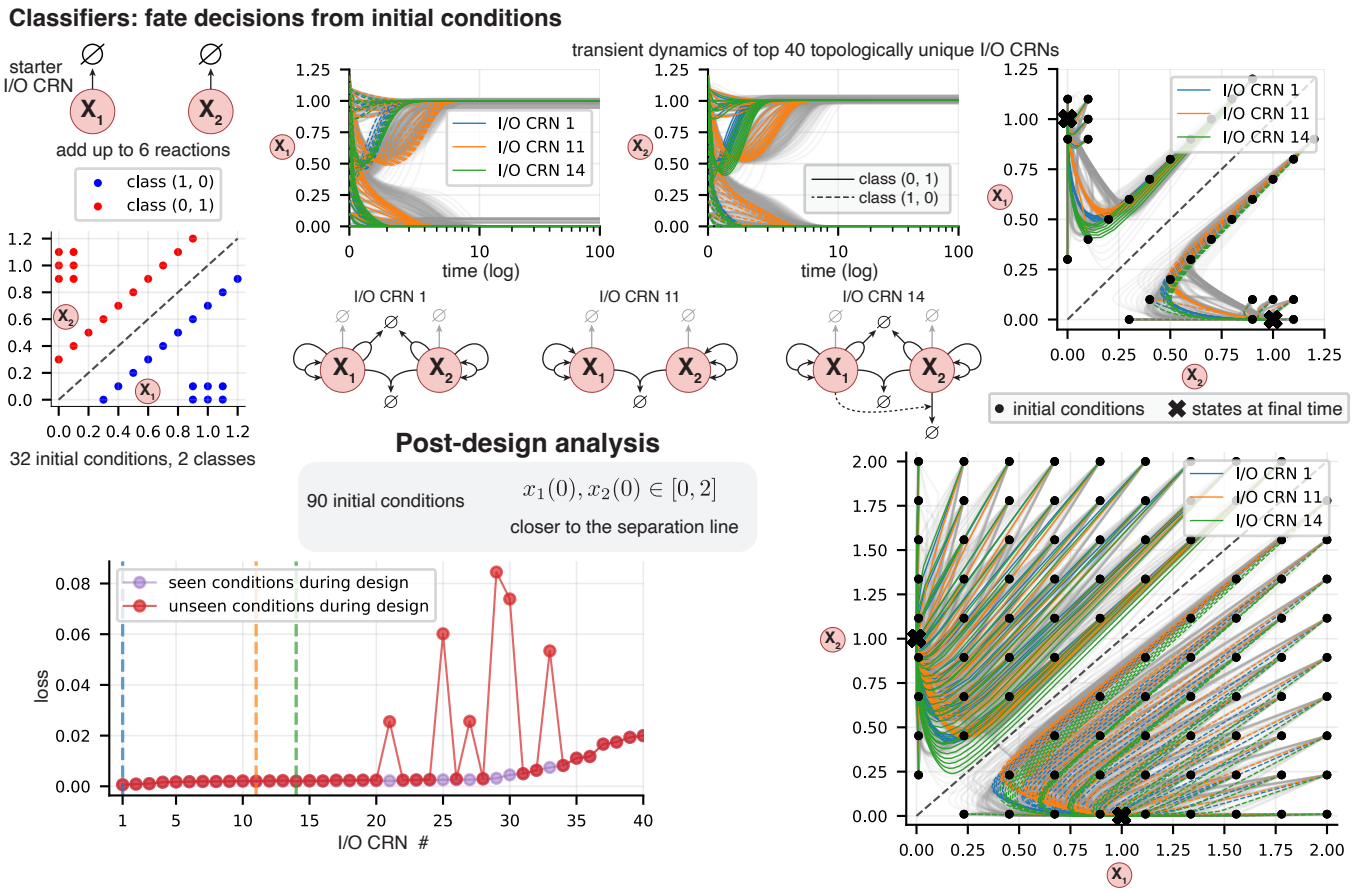
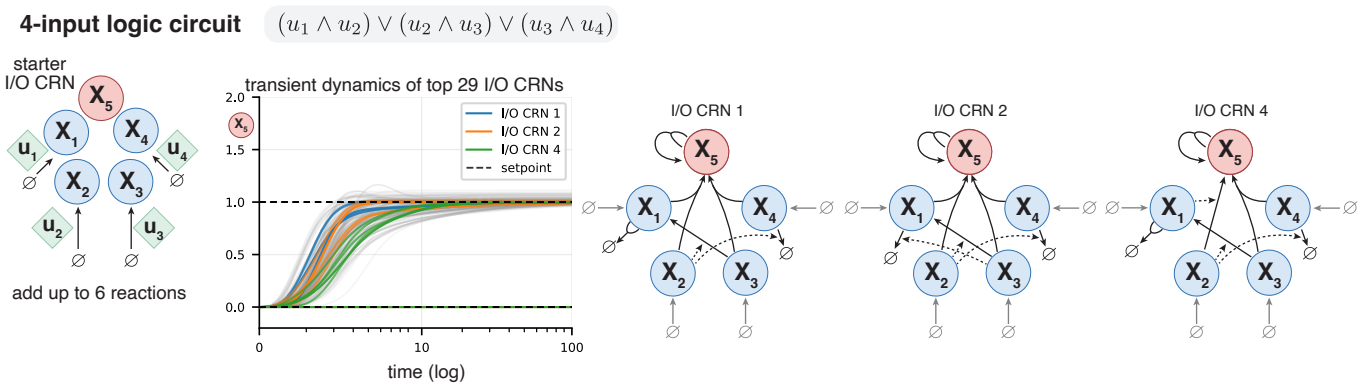


Figure 6: **Classifiers and logic circuits. Molecular classifiers from initial-condition–dependent fate decisions.** GenAI-Net is tasked with generating I/O CRNs that *classify* the initial state of a two-species system into one of two discrete outcomes. The starter I/O CRN (top left) contains two output species, $\mathbf{X_1}$ and $\mathbf{X_2}$ (red), and no external inputs, so classification is driven solely by the network's intrinsic nonlinear dynamics. A set of 32 initial conditions in the $\mathbf{X_1}$–$\mathbf{X_2}$ plane (left) is partitioned into two classes (blue: class $(1, 0)$; red: class $(0, 1)$) corresponding to desired long-time fates in which one species is high while the other is low (diagonal decision boundary shown). From the starter I/O CRN, GenAI-Net generates candidate I/O CRNs by appending up to 6 doubly bimolecular reactions with mass-action kinetics, and optimizing a task loss that penalizes incorrect assignment of initial conditions to the target fate basins (see Methods for the the loss function). The time-course plots (top center) show $\mathbf{X_1}$ (left) and $\mathbf{X_2}$ (right) trajectories versus log time for the top 40 topologically unique generated I/O CRNs (thin gray curves), with three representative solutions highlighted (colored; I/O CRNs 1, 11, and 14). For each highlighted I/O CRN, trajectories are shown across the full set of labeled initial conditions, demonstrating separation into two attractors consistent with the target classes (solid versus dashed curves). The phase-plane visualization (top right) summarizes the same simulations in the $\mathbf{X_1}$–$\mathbf{X_2}$ plane, where points indicate trajectories from the labeled initial conditions and the two terminal regions correspond to the two classes (crosses); example reaction topologies for the highlighted I/O CRNs are shown in the center. *Post-design analysis* (bottom): to test generalization beyond the initial conditions seen during the design process, the generated I/O CRNs are evaluated on an expanded set of 90 initial conditions, some of which are concentrated near the decision boundary. The loss-versus-network index plot contrasts initial conditions seen during design (purple) with unseen initial conditions (red), and the phase-plane plot shows the resulting terminal states, highlighting how the learned basin structure extrapolates to nearby initial conditions not seen during the design.



u_1	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
u_2	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
u_3	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
u_4	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
Target	0.00	0.00	0.00	1.00	0.00	0.00	1.00	1.00	0.00	0.00	0.00	1.00	1.00	1.00	1.00	1.00
Output	**0.00**	**0.00**	**0.00**	**1.00**	**0.00**	**0.00**	**0.99**	**0.99**	**0.00**	**0.00**	**0.00**	**1.00**	**0.99**	**0.99**	**1.00**	**1.01**

I/O CRN 1

u_1	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
u_2	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
u_3	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
u_4	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
Target	0.00	0.00	0.00	1.00	0.00	0.00	1.00	1.00	0.00	0.00	0.00	1.00	1.00	1.00	1.00	1.00
Output	**0.00**	**0.00**	**0.00**	**1.00**	**0.00**	**0.00**	**0.99**	**0.99**	**0.00**	**0.00**	**0.00**	**1.00**	**1.00**	**1.00**	**1.00**	**1.02**

I/O CRN 2

u_1	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
u_2	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
u_3	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
u_4	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
Target	0.00	0.00	0.00	1.00	0.00	0.00	1.00	1.00	0.00	0.00	0.00	1.00	1.00	1.00	1.00	1.00
Output	**0.00**	**0.00**	**0.00**	**0.99**	**0.00**	**0.00**	**0.99**	**0.99**	**0.00**	**0.00**	**0.00**	**0.99**	**0.99**	**0.99**	**0.99**	**1.00**

I/O CRN 4

Figure 7: **Four-input irreducible logic circuit discovery.** GenAI-Net begins with a starter I/O CRN (top left) in which four external inputs u_1–u_4 act on corresponding input species $\mathbf{X_1}$–$\mathbf{X_4}$ (blue) to regulate an output species (red). From this starter I/O CRN, GenAI-Net generates candidate I/O CRNs by appending additional reactions and optimizing a loss (see Methods) that rewards correct realization of the target 4-input irreducible Boolean function $(u_1 \wedge u_2) \vee (u_2 \wedge u_3) \vee (u_3 \wedge u_4)$. The time-course panel shows the output response versus time for the discovered solution set (thin gray trajectories), with three representative solutions highlighted (colored curves) and the low/high digital setpoints indicated by horizontal dashed lines. Example I/O CRN topologies for the highlighted networks are shown to the right, together with their truth tables (bottom), demonstrating high-precision binarization of the output across all 16 input combinations.

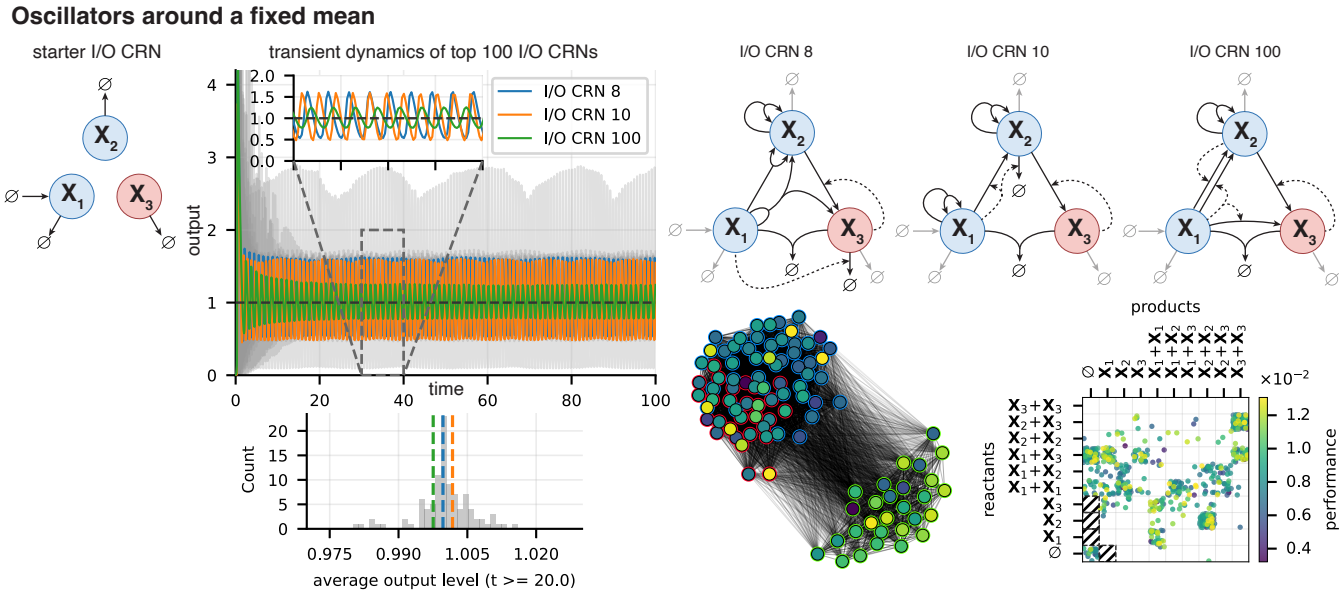


Figure 8: **Molecular oscillators. Oscillators around a fixed mean.** GenAI-Net begins with a starter I/O CRN (left) with three species, $\mathbf{X_1}$, $\mathbf{X_2}$ (blue), and output $\mathbf{X_3}$ (red), and no external input. From this template, GenAI-Net generates candidate I/O CRNs by appending up to 6 doubly bimolecular reactions, following mass-action kinetics, and optimizing a task objective that rewards sustained oscillations while constraining the temporal average output to a fixed mean equal to one (see loss function in Methods for more details). The center panel shows output trajectories for the top 100 topologically unique generated I/O CRNs (thin gray curves), with three representative solutions highlighted (colored; I/O CRNs 8, 10, and 100). The histogram (bottom left) summarizes the distribution of time-averaged output levels across the 100 solutions (computed over $t \geq 20$), illustrating concentration near the target mean. Example I/O CRN topologies for the highlighted networks are shown to the right. The graph visualization (bottom center) summarizes topological diversity across the solution set, and the reactant–product incidence map (bottom right) summarizes reaction usage across generated solutions.

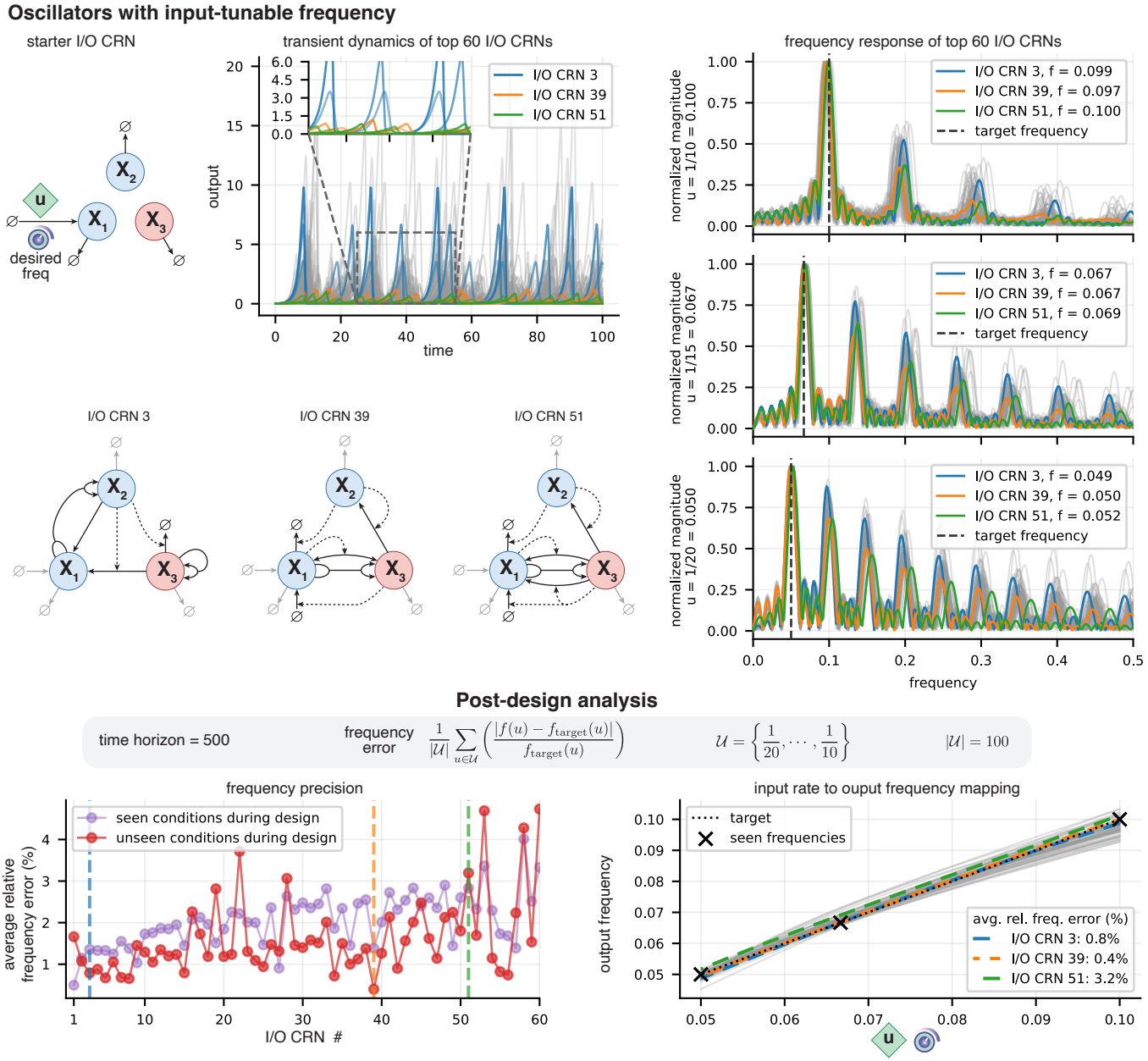


Figure 9: **Oscillators with input-tunable frequency.** A related starter I/O CRN introduces an external input u_1 (green) that specifies a *target oscillation frequency*. GenAI-Net again appends up to 6 reactions and optimizes a frequency-tracking objective (see loss function in Methods for details), producing I/O CRNs whose oscillation frequency is tuned by u_1 while maintaining oscillatory behavior. The left-center plot shows time-domain output trajectories for the top 60 topologically unique solutions (gray), with three representative networks highlighted (I/O CRNs 1, 2, and 51). Frequency-domain behavior is summarized on the right: for three example input settings, the normalized magnitude spectra of the highlighted networks are shown, with dashed lines indicating the target frequencies; legend annotations report the achieved dominant frequencies f for each I/O CRN. Example network topologies for the highlighted solutions are shown beneath the time-domain plot, and the topological diversity graph and reactant–product incidence map (bottom) summarize structural diversity and reaction usage across the generated oscillator solution set. **Post-design testing beyond seen conditions.** To assess generalization, the top I/O CRNs from (B) are evaluated on a dense set of 100 input frequencies spanning the design range (schematic, top), using a longer simulation horizon (t_f = 500). The left panel compares the average relative frequency error on the input frequencies used during design (purple) versus additional frequencies not used during design (red) across the top 60 I/O CRNs. Using the longer horizon improves frequency estimation and yields higher precision, including on unseen input frequencies. The right panel plots output frequency versus input frequency for all top 60 I/O CRNs (gray) and the representative networks (colored), showing close agreement with the identity line and reporting the average relative error for each highlighted I/O CRN.

References and Notes

1. M. Jinek, *et al.*, A programmable dual-RNA–guided DNA endonuclease in adaptive bacterial immunity. *science* **337** (6096), 816–821 (2012).

2. L. Cong, *et al.*, Multiplex genome engineering using CRISPR/Cas systems. *Science* **339** (6121), 819–823 (2013).

3. S. Kosuri, G. M. Church, Large-scale de novo DNA synthesis: technologies and applications. *Nature methods* **11** (5), 499–507 (2014).

4. A. Hoose, R. Vellacott, M. Storch, P. S. Freemont, M. G. Ryadnov, DNA synthesis technologies to close the gene writing gap. *Nature Reviews Chemistry* **7** (3), 144–161 (2023).

5. M. Chehelgerdi, *et al.*, Comprehensive review of CRISPR-based gene editing: mechanisms, challenges, and applications in cancer therapy. *Molecular cancer* **23** (1), 9 (2024).

6. M. B. Elowitz, S. Leibler, A synthetic oscillatory network of transcriptional regulators. *Nature* **403** (6767), 335–338 (2000).

7. T. S. Gardner, C. R. Cantor, J. J. Collins, Construction of a genetic toggle switch in Escherichia coli. *Nature* **403** (6767), 339–342 (2000).

8. A. Becskei, L. Serrano, Engineering stability in gene networks by autoregulation. *Nature* **405** (6786), 590–593 (2000).

9. S. K. Aoki, *et al.*, A universal biomolecular integral feedback controller for robust perfect adaptation. *Nature* **570** (7762), 533–537 (2019).

10. D. L. Porter, B. L. Levine, M. Kalos, A. Bagg, C. H. June, Chimeric antigen receptor–modified T cells in chronic lymphoid leukemia. *New England Journal of Medicine* **365** (8), 725–733 (2011).

11. K. T. Roybal, *et al.*, Precision tumor recognition by T cells with combinatorial antigen-sensing circuits. *Cell* **164** (4), 770–779 (2016).

12. A. S. Khalil, J. J. Collins, Synthetic biology: applications come of age. *Nature Reviews Genetics* **11** (5), 367–379 (2010).

13. C. A. Voigt, Synthetic biology 2020–2030: six commercially-available products that are changing our world. *Nature Communications* **11** (1), 6379 (2020).

14. D. K. Agrawal, R. Marshall, V. Noireaux, E. D. Sontag, In vitro implementation of robust gene regulation in a synthetic biomolecular integral controller. *Nature communications* **10** (1), 5760 (2019).

15. C. Briat, A. Gupta, M. Khammash, Antithetic integral feedback ensures robust perfect adaptation in noisy biomolecular networks. *Cell systems* **2** (1), 15–26 (2016).

16. C. Briat, C. Zechner, M. Khammash, Design of a synthetic integral feedback circuit: dynamic analysis and DNA implementation. *ACS synthetic biology* **5** (10), 1108–1116 (2016).

17. X. Y. Ni, T. Drengstig, P. Ruoff, The control of the controller: molecular mechanisms for robust perfect adaptation and temperature compensation. *Biophysical journal* **97** (5), 1244–1253 (2009).

18. R. D. Jones, *et al.*, Robust and tunable signal processing in mammalian cells via engineered covalent modification cycles. *Nature communications* **13** (1), 1720 (2022).

19. S. Anastassov, M. Filo, C.-H. Chang, M. Khammash, A cybergenetic framework for engineering intein-mediated integral feedback control systems. *Nature Communications* **14** (1), 1337 (2023).

20. T. Frei, C.-H. Chang, M. Filo, A. Arampatzis, M. Khammash, A genetic mammalian proportional–integral feedback control circuit for robust and precise gene regulation. *Proceedings of the National Academy of Sciences* **119** (00), e2122132119 (2022).

21. D. Salzano, *et al.*, In Vivo Multicellular Feedback Control in Synthetic Microbial Consortia. *ACS Synthetic Biology* (2025).

22. C.-H. Chang, *et al.*, Engineering Cybergenetic Cell-Based Therapies. *bioRxiv* pp. 2026–01 (2026).

23. M. Filo, S. Kumar, M. Khammash, A hierarchy of biomolecular proportional-integral-derivative feedback controllers for robust perfect adaptation and dynamic performance. *Nature communications* **13** (1), 2119 (2022).

24. M. Chevalier, M. Gómez-Schiavon, A. H. Ng, H. El-Samad, Design and analysis of a proportional-integral-derivative controller with biological molecules. *Cell systems* **9** (4), 338–353 (2019).

25. E. Alexis, L. Cardelli, A. Papachristodoulou, On the design of a PID bio-controller with set point weighting and filtered derivative action. *IEEE Control Systems Letters* **6**, 3134–3139 (2022).

26. V. Martinelli, D. Fiore, D. Salzano, M. di Bernardo, Multicellular PID control for robust regulation of biological processes. *Journal of the Royal Society Interface* **22** (222), 20240583 (2025).

27. N. M. Paulino, M. Foo, J. Kim, D. G. Bates, PID and state feedback controllers using DNA strand displacement reactions. *IEEE Control Systems Letters* **3** (4), 805–810 (2019).

28. M. Whitby, *et al.*, PID control of biochemical reaction networks. *IEEE Transactions on Automatic Control* **67** (2), 1023–1030 (2021).

29. S. Modi, S. Dey, A. Singh, Noise suppression in stochastic genetic circuits using PID controllers. *PLoS Computational Biology* **17** (7), e1009249 (2021).

30. C. C. Samaniego, E. Franco, Ultrasensitive molecular controllers for quasi-integral feedback. *Cell Systems* **12** (3), 272–288 (2021).

31. B. Kell, R. Ripsman, A. Hilfinger, Noise properties of adaptation-conferring biochemical control modules. *Proceedings of the National Academy of Sciences* **120** (38), e2302016120 (2023).

32. M. Filo, M. Hou, M. Khammash, A hidden proportional feedback mechanism underlies enhanced dynamic performance and noise rejection in sensor-based antithetic integral control. *bioRxiv* pp. 2023–04 (2023).

33. M. Filo, A. Gupta, M. Khammash, Anti-windup strategies for biomolecular control systems facilitated by model reduction theory for sequestration networks. *Science Advances* **10** (34), eadl5439 (2024).

34. E. J. Hancock, D. A. Oyarzún, Stabilization of antithetic control via molecular buffering. *Journal of The Royal Society Interface* **19** (188), 20210762 (2022).

35. K. Oishi, E. Klavins, Biomolecular implementation of linear I/O systems. *IET systems biology* **5** (4), 252–260 (2011).

36. C. Zechner, G. Seelig, M. Rullan, M. Khammash, Molecular circuits for dynamic noise filtering. *Proceedings of the National Academy of Sciences* **113** (17), 4729–4734 (2016).

37. K. M. Cherry, L. Qian, Supervised learning in DNA neural networks. *Nature* pp. 1–9 (2025).

38. G. Seelig, D. Soloveichik, D. Y. Zhang, E. Winfree, Enzyme-free nucleic acid logic circuits. *science* **314** (5805), 1585–1588 (2006).

39. T. S. Moon, C. Lou, A. Tamsir, B. C. Stanton, C. A. Voigt, Genetic programs constructed from layered logic gates in single cells. *Nature* **491** (7423), 249–253 (2012).

40. S. A. Malekpour, M. Shahdoust, R. Aghdam, M. Sadeghi, wpLogicNet: logic gate and structure inference in gene regulatory networks. *Bioinformatics* **39** (2), btad072 (2023).

41. Z. Chen, *et al.*, A synthetic protein-level neural network in mammalian cells. *Science* **386** (6727), 1243–1250 (2024).

42. K. M. Cherry, L. Qian, Scaling up molecular pattern recognition with DNA-based winner-take-all neural networks. *Nature* **559** (7714), 370–376 (2018).

43. M. Vasić, C. Chalk, A. Luchsinger, S. Khurshid, D. Soloveichik, Programming and training rate-independent chemical reaction networks. *Proceedings of the National Academy of Sciences* **119** (24), e2111552119 (2022).

44. L. Qian, E. Winfree, J. Bruck, Neural network computation with DNA strand displacement cascades. *nature* **475** (7356), 368–372 (2011).

45. S. Okumura, *et al.*, Nonlinear decision-making with enzymatic neural networks. *Nature* **610** (7932), 496–501 (2022).

46. D. F. Anderson, B. Joshi, A. Deshpande, On reaction network implementations of neural networks. *Journal of the Royal Society Interface* **18** (177), 20210031 (2021).

47. Y. Fan, X. Zhang, C. Gao, D. Dochain, Automatic Implementation of Neural Networks through Reaction Networks—Part I: Circuit Design and Convergence Analysis. *IEEE Transactions on Automatic Control* (2025).

48. A. Moorman, C. C. Samaniego, C. Maley, R. Weiss, A dynamical biomolecular neural network, in *2019 IEEE 58th conference on decision and control (CDC)* (IEEE) (2019), pp. 1797–1802.

49. C. C. Samaniego, E. Wallace, F. Blanchini, E. Franco, G. Giordano, Neural networks built from enzymatic reactions can operate as linear and nonlinear classifiers, in *2024 IEEE 63rd Conference on Decision and Control (CDC)* (IEEE) (2024), pp. 6292–6297.

50. W. Ma, A. Trusina, H. El-Samad, W. A. Lim, C. Tang, Defining network topologies that can achieve biochemical adaptation. *Cell* **138** (4), 760–773 (2009).

51. T. W. Hiscock, Adapting machine-learning algorithms to design gene circuits. *BMC bioinformatics* **20** (1), 214 (2019).

52. N. Rossi, A. Gupta, M. Khammash, SynthEvo: A Gradient-Guided Evolutionary Approach for Synthetic Circuit Design, in *2024 IEEE 63rd Conference on Decision and Control (CDC)* (IEEE) (2024), pp. 6298–6304.

53. G. Rodrigo, J. Carrera, A. Jaramillo, Genetdes: automatic design of transcriptional networks. *Bioinformatics* **23** (14), 1857–1858 (2007).

54. L. T. Tatka, L. P. Smith, H. M. Sauro, Speciated evolution of oscillatory mass-action chemical reaction networks. *bioRxiv* (2025).

55. S. Paladugu, *et al.*, In silico evolution of functional modules in biochemical networks. *IEE Proceedings-Systems Biology* **153** (4), 223–235 (2006).

56. P. S. Bhamidipati, M. Thomson, Designing biochemical circuits with tree search. *bioRxiv* pp. 2025–01 (2025).

57. M. Rybiński, S. Möller, M. Sunnåker, C. Lormeau, J. Stelling, TopoFilter: a MATLAB package for mechanistic model identification in systems biology. *BMC bioinformatics* **21** (1), 34 (2020).

58. M. Kobiela, D. A. Oyarzún, M. U. Gutmann, Risk-averse optimization of genetic circuits under uncertainty. *bioRxiv* pp. 2024–11 (2024).

59. C. Merzbacher, O. Mac Aodha, D. A. Oyarzún, Bayesian optimization for design of multiscale biological circuits. *ACS synthetic biology* **12** (7), 2073–2082 (2023).

60. O. Gallup, H. Steel, Generative design of synthetic gene circuits for functional and evolutionary properties. *bioRxiv* pp. 2025–09 (2025).

61. K. Rai, *et al.*, Ultra-high-throughput mapping of genetic design space. *Nature* pp. 1–10 (2026).

62. S. Palacios, J. J. Collins, D. Del Vecchio, Machine learning for synthetic gene circuit engineering. *Current Opinion in Biotechnology* **92**, 103263 (2025).

63. K. Rai, Y. Wang, R. W. O'Connell, A. B. Patel, C. J. Bashor, Using machine learning to enhance and accelerate synthetic biology. *Current Opinion in Biomedical Engineering* **31**, 100553 (2024).

64. M. Filo, N. Rossi, F. Zhou, GenAI-Net (2026), doi:10.5281/zenodo.20485800, https://github.com/Maurice-Filo/GenAI-Net.

65. D. T. Gillespie, Exact stochastic simulation of coupled chemical reactions. *The journal of physical chemistry* **81** (25), 2340–2361 (1977).

66. M. Filo, C.-H. Chang, M. Khammash, Biomolecular feedback controllers: from theory to applications. *Current Opinion in Biotechnology* **79**, 102882 (2023).

67. M. E. Kotas, R. Medzhitov, Homeostasis, inflammation, and disease susceptibility. *Cell* **160** (5), 816–827 (2015).

68. T. O'Leary, Homeostasis, failure of homeostasis and degenerate ion channel regulation. *Current Opinion in Physiology* **2**, 129–138 (2018).

69. M. H. Khammash, Perfect adaptation in biology. *Cell Systems* **12** (6), 509–521 (2021).

70. T. Frei, M. Khammash, Adaptive circuits in synthetic biology. *Current Opinion in Systems Biology* **28**, 100399 (2021).

71. F. Xiao, J. C. Doyle, Robust perfect adaptation in biomolecular reaction networks, in *2018 IEEE conference on decision and control (CDC)* (IEEE) (2018), pp. 4345–4352.

72. T. Drengstig, X. Ni, K. Thorsen, I. Jolma, P. Ruoff, Robust adaptation and homeostasis by autocatalysis. *The Journal of Physical Chemistry B* **116** (18), 5355–5363 (2012).

73. Y. Qian, D. Del Vecchio, Realizing 'integral control'in living cells: how to overcome leaky integration due to dilution? *Journal of The Royal Society Interface* **15** (139), 20170902 (2018).

74. H. Steel, A. Papachristodoulou, Low-burden biological feedback controllers for near-perfect adaptation. *ACS synthetic biology* **8** (10), 2212–2219 (2019).

75. C. L. Kelly, *et al.*, Synthetic negative feedback circuits using engineered small RNAs. *Nucleic acids research* **46** (18), 9875–9889 (2018).

76. A. M. Zand, S. Anastassov, T. Frei, M. Khammash, Multi-Layer Autocatalytic Feedback Enables Integral Control Amidst Resource Competition and Across Scales. *ACS Synthetic Biology* **14** (4), 1041–1061 (2025).

77. A. M. Zand, A. Gupta, M. Khammash, Cascaded antithetic integral feedback for enhanced stability and performance. *IEEE Control Systems Letters* (2024).

78. H.-H. Huang, Y. Qian, D. Del Vecchio, A quasi-integral controller for adaptation of genetic modules to variable ribosome demand. *Nature communications* **9** (1), 5415 (2018).

79. A. Mallozzi, V. Fusco, F. Ragazzini, D. di Bernardo, The CRISPRaTOR: a biomolecular circuit for Automatic Gene Regulation in Mammalian Cells with CRISPR technology. *bioRxiv* (2024).

80. Y. Zhang, S. Zhang, CRISPR perfect adaptation for robust control of cellular immune and apoptotic responses. *Nucleic Acids Research* **52** (16), 10005–10016 (2024).

81. A. Gupta, M. Khammash, Universal structural requirements for maximal robust perfect adaptation in biomolecular networks. *Proceedings of the National Academy of Sciences* **119** (43), e2207802119 (2022).

82. A. Gupta, M. Khammash, The internal model principle for biomolecular control theory. *IEEE Open Journal of Control Systems* **2**, 63–69 (2023).

83. Y. Hirono, A. Gupta, M. Khammash, Rethinking robust adaptation: Characterization of structural mechanisms for biochemical network robustness through topological invariants. *PRX Life* **3** (1), 013017 (2025).

84. R. P. Araujo, L. A. Liotta, The topological requirements for robust perfect adaptation in networks of any size. *Nature communications* **9** (1), 1757 (2018).

85. M. Smart, S. Y. Shvartsman, M. Mönnigmann, Minimal motifs for habituating systems. *Proceedings of the National Academy of Sciences* **121** (41), e2409330121 (2024).

86. L. Eckert, *et al.*, Biochemically plausible models of habituation for single-cell learning. *Current Biology* **34** (24), 5646–5658 (2024).

87. H. H. McAdams, A. Arkin, Stochastic mechanisms in gene expression. *Proceedings of the National Academy of Sciences* **94** (3), 814–819 (1997).

88. I. Lestas, G. Vinnicombe, J. Paulsson, Fundamental limits on the suppression of molecular fluctuations. *Nature* **467** (7312), 174–178 (2010).

89. C. Briat, A. Gupta, M. Khammash, Antithetic proportional-integral feedback for reduced variance and improved control performance of stochastic reaction networks. *Journal of The Royal Society Interface* **15** (143), 20180079 (2018).

90. D. Lim, *et al.*, Toward single-cell control: noise-robust perfect adaptation in biomolecular systems. *Nature Communications* (2025).

91. E. Bier, E. M. De Robertis, BMP gradients: A paradigm for morphogen-mediated developmental patterning. *Science* **348** (6242), aaa5838 (2015).

92. I. Otero-Muras, P. Yordanov, J. Stelling, Chemical reaction network theory elucidates sources of multistability in interferon signaling. *PLoS computational biology* **13** (4), e1005454 (2017).

93. J. Kim, K. S. White, E. Winfree, Construction of an in vitro bistable circuit from synthetic transcriptional switches. *Molecular systems biology* **2** (1), 68 (2006).

94. J. Santos-Moreno, E. Tasiudi, J. Stelling, Y. Schaerli, Multistable and dynamic CRISPRi-based synthetic circuits. *Nature communications* **11** (1), 2746 (2020).

95. M. Wu, *et al.*, Engineering of regulated stochastic cell fate determination. *Proceedings of the National Academy of Sciences* **110** (26), 10610–10615 (2013).

96. D. A. Oyarzún, M. Chaves, Design of a bistable switch to control cellular uptake. *Journal of The Royal Society Interface* **12** (113), 20150618 (2015).

97. D. Siegal-Gaskins, E. Grotewold, G. D. Smith, The capacity for multistability in small gene regulatory networks. *BMC systems biology* **3** (1), 96 (2009).

98. M. H. Vitaterna, *et al.*, Mutagenesis and mapping of a mouse gene, Clock, essential for circadian behavior. *Science* **264** (5159), 719–725 (1994).

99. T. A. Bargiello, M. W. Young, Molecular genetics of a biological clock in Drosophila. *Proceedings of the National Academy of Sciences* **81** (7), 2142–2146 (1984).

100. N. Geva-Zatorsky, *et al.*, Oscillations and variability in the p53 system. *Molecular systems biology* **2** (1), 2006–0033 (2006).

101. Y. Chen, J. K. Kim, A. J. Hirning, K. Josić, M. R. Bennett, Emergent genetic oscillations in a synthetic microbial consortium. *Science* **349** (6251), 986–989 (2015).

102. F. A. Chandra, G. Buzi, J. C. Doyle, Glycolytic oscillations and limits on robust efficiency. *science* **333** (6039), 187–192 (2011).

103. M. Tigges, N. Dénervaud, D. Greber, J. Stelling, M. Fussenegger, A synthetic low-frequency mammalian oscillator. *Nucleic acids research* **38** (8), 2702–2711 (2010).

104. B. Novák, J. J. Tyson, Design principles of biochemical oscillators. *Nature reviews Molecular cell biology* **9** (12), 981–991 (2008).

105. J. Kim, E. Winfree, Synthetic in vitro transcriptional oscillators. *Molecular systems biology* **7** (1), 465 (2011).

106. J. Stricker, *et al.*, A fast, robust and tunable synthetic gene oscillator. *Nature* **456** (7221), 516–519 (2008).

107. B.-W. Qin, L. Zhao, W. Lin, A frequency-amplitude coordinator and its optimal energy consumption for biological oscillators. *Nature Communications* **12** (1), 5894 (2021).

108. J. K. Kim, D. B. Forger, A mechanism for robust circadian timekeeping via stoichiometric balance. *Molecular systems biology* **8**, 630 (2012).

109. E. M. Jeong, Y. M. Song, J. K. Kim, Combined multiple transcriptional repression mechanisms generate ultrasensitivity and oscillations. *Interface Focus* **12** (3) (2022).

110. A. Vaswani, *et al.*, Attention is all you need. *Advances in neural information processing systems* **30** (2017).

111. D. Zhang, *et al.*, Let the flows tell: Solving graph combinatorial problems with gflownets. *Advances in neural information processing systems* **36**, 11952–11969 (2023).

112. J. Ho, A. Jain, P. Abbeel, Denoising Diffusion Probabilistic Models. *CoRR* **abs/2006.11239** (2020), https://arxiv.org/abs/2006.11239.

113. J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).

114. R. J. Williams, Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning* **8** (3), 229–256 (1992).

115. Y. Chow, M. Ghavamzadeh, Algorithms for CVaR optimization in MDPs. *Advances in neural information processing systems* **27** (2014).

116. O. Delalleau, M. Peter, E. Alonso, A. Logut, Discrete and continuous action representation for practical RL in video games. *arXiv preprint arXiv:1912.11077* (2019).

117. M. Geist, B. Scherrer, O. Pietquin, A theory of regularized markov decision processes, in *International conference on machine learning* (PMLR) (2019), pp. 2160–2169.

118. J. Oh, Y. Guo, S. Singh, H. Lee, Self-imitation learning, in *International conference on machine learning* (PMLR) (2018), pp. 3878–3887.

Funding: This work is supported by the Swiss State Secretariat for Education, Research and Innovation (SERI). Z.F. is supported by the National Natural Science Foundation of China (Grant No. 12501699) and the Open Foundation of the State Key Laboratory of Mathematical Sciences (Grant No. SLMS-2025-KFKT-TD-01).

Author contributions: MF, NR, and ZF conceived the study in consultation with MK. MF, NR and ZF developed the framework and performed the computational analyses. MK provided overall scientific direction, supervised the research, and secured funding. All authors wrote the paper.

Competing interests: All authors declare they have no competing interests.

Data, code and materials availability: All the code to reproduce the results of this work, and to explore other applications is available in the following Github repository `https://github.com/Maurice-Filo/GenAI-Net` with an easy-to-use programmatic user interface and detailed documentation `https://maurice-filo.github.io/GenAI-Net/` and list of generated CRNs.

Supplementary materials

Figs. S1 to S5

Supplementary Materials for
GenAI-Net: A Generative AI Framework for Automated Biomolecular Network Design

Maurice Filo[†], Nicolò Rossi[†], Zhou Fang[†], Mustafa Khammash[*]
[*]Corresponding author. Email: mustafa.khammash@bsse.ethz.ch
[†]These authors contributed equally to this work.

This PDF file includes:

Figures S1 to S5

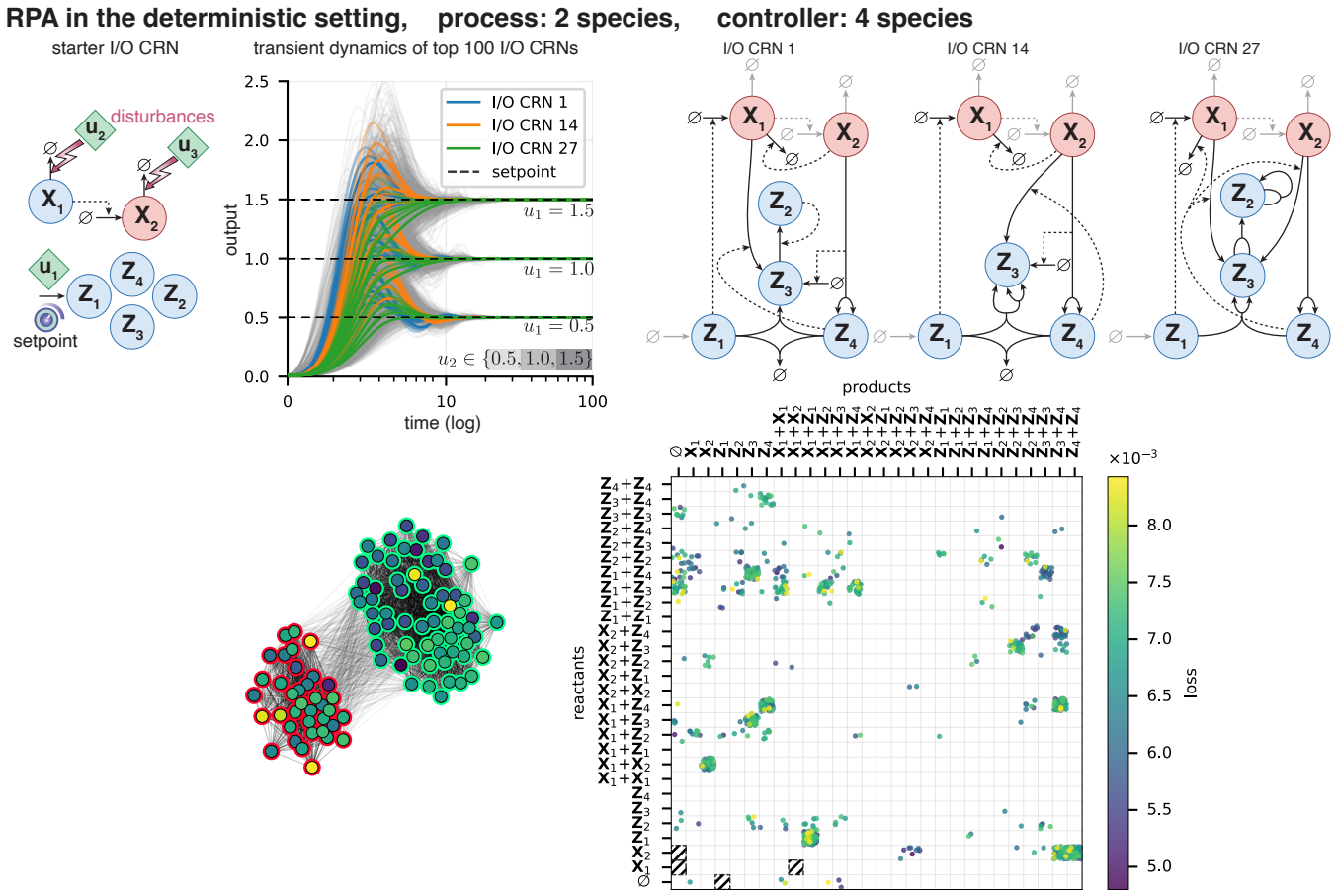


Figure S1: **Deterministic RPA with 6 species.** GenAI-Net scales to a higher-dimensional setting with two process species, the regulated output $\mathbf{X_2}$ (red) and an additional species $\mathbf{X_1}$, together with four controller species $\mathbf{Z_1}$–$\mathbf{Z_4}$ (blue). The setpoint is specified by u_1, while disturbances u_2 and u_3 modulate degradation of $\mathbf{X_1}$ and $\mathbf{X_2}$, respectively. Starting from this multi-species starter I/O CRN, GenAI-Net generates candidate I/O CRNs by appending up to eight doubly bimolecular mass-action reactions and optimizing the same setpoint-tracking objective as in Fig. 4A. The center plot shows output trajectories for all generated solutions (gray), with three representative networks highlighted (I/O CRNs 1, 14, and 27) and their corresponding topologies shown to the right. The graph visualization and reactant–product incidence map summarize topological diversity and reaction usage across the generated collection, as in Fig. 4A.

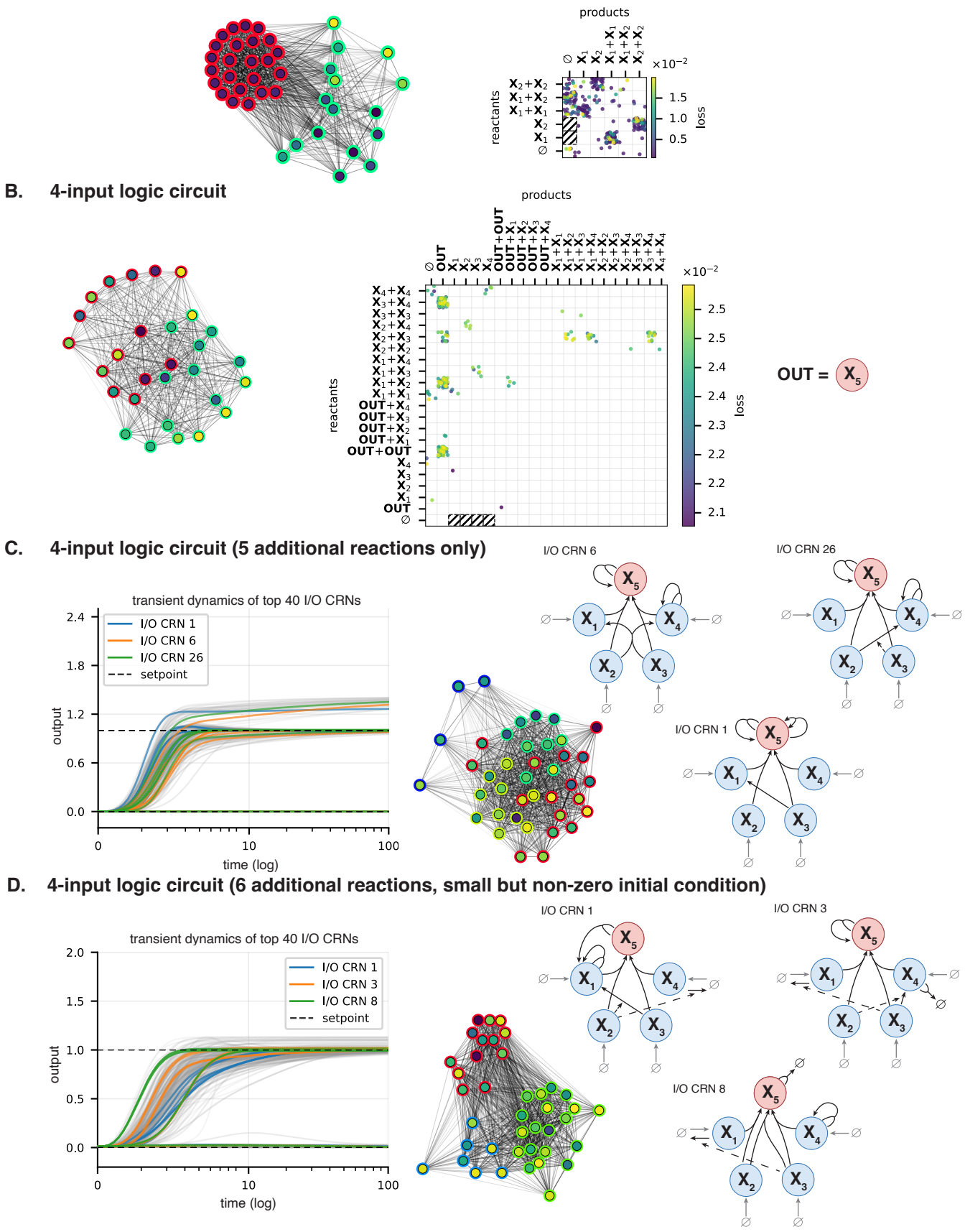


Figure S2: **Classifiers and logic circuits.** The figure caption is on the next page.

Figure S2: **Classifiers and logic circuits. (a)** and **(b)**: The graph visualization and reactant–product incidence map summarizing topological diversity and reaction usage across the generated collections of Fig. 6 and 7 respectively. **(c)** Restricting the search to at most 5 appended reactions (instead of 6 as in Fig. 7), circuits that remain correct under thresholding but exhibit a more analog implementation of the logic; in particular, highlighted examples show a noticeable overshoot above the nominal "high" level in the all-inputs-high condition, in contrast to the cleaner saturation obtained in Fig. 7 when allowing one additional reaction. **(d)** This panel is the same as Fig. 7 with only one difference: the initial conditions are not zero to avoid exploiting unstable fixed points at the origin. GenAI-Net also succeeded in generating diverse I/O CRNs under these conditions.

A. Reaction library with Hill kinetics (433 reactions)

$\varnothing \xrightarrow{} X_k \quad \alpha_0 + \alpha_1 \frac{(x_i/\kappa)^n}{1+(x_i/\kappa)^n}$

$X_i \dashv X_k \quad \alpha_0 + \alpha_1 \frac{1}{1+(x_i/\kappa)^n}$

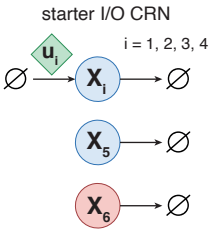
$X_i \xrightarrow{} X_k \quad \alpha_0 + \alpha_1 \frac{(x_i/\kappa_1)^{n_1}}{1+(x_i/\kappa_1)^{n_1}} \frac{(x_j/\kappa_2)^{n_2}}{1+(x_j/\kappa_2)^{n_2}}$

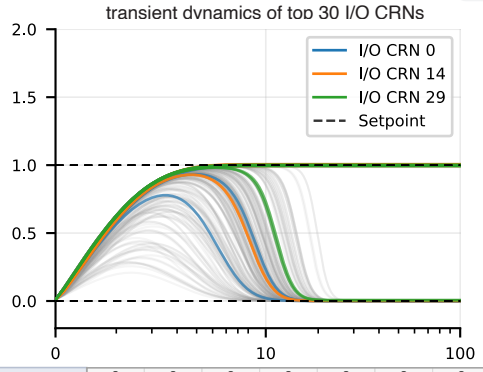
$X_i \dashv X_k \quad \alpha_0 + \alpha_1 \frac{1}{1+(x_i/\kappa_1)^{n_1}} \frac{1}{1+(x_j/\kappa_2)^{n_2}}$

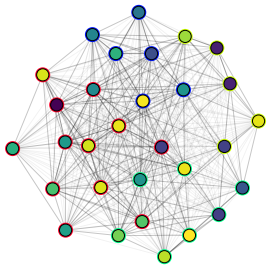
$\varnothing \xrightarrow{} \varnothing \qquad X_i \dashv X_k \quad \alpha_0 + \alpha_1 \frac{(x_i/\kappa_1)^{n_1}}{1+(x_i/\kappa_1)^{n_1}} \frac{1}{1+(x_j/\kappa_2)^{n_2}}$

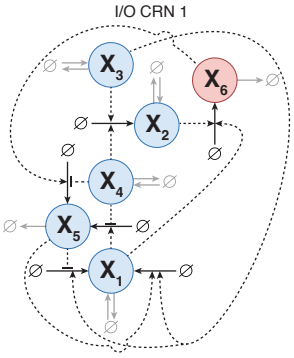
$i,j,k \in \{1,2,3,4,5,6\}$

B. 4-input logic circuit (6 additional reactions) - Hill function $(u_1 \wedge u_2) \vee (u_2 \wedge u_3) \vee (u_3 \wedge u_4)$

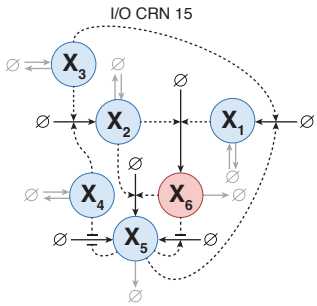




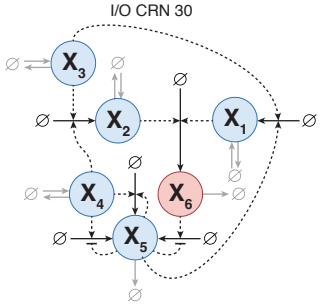




u_1	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
u_2	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
u_3	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
u_4	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
Target	0.00	0.00	0.00	1.00	0.00	0.00	1.00	1.00	0.00	0.00	0.00	1.00	1.00	1.00	1.00	1.00
Output	**0.00**	**0.00**	**0.00**	**1.00**	**0.00**	**0.00**	**1.00**	**1.00**	**0.00**	**0.00**	**0.00**	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**



u_1	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
u_2	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
u_3	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
u_4	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
Target	0.00	0.00	0.00	1.00	0.00	0.00	1.00	1.00	0.00	0.00	0.00	1.00	1.00	1.00	1.00	1.00
Output	**0.00**	**0.00**	**0.00**	**1.00**	**0.00**	**0.00**	**1.00**	**1.01**	**0.00**	**0.00**	**0.00**	**1.00**	**0.99**	**0.99**	**1.00**	**1.01**



u_1	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
u_2	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
u_3	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
u_4	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
Target	0.00	0.00	0.00	1.00	0.00	0.00	1.00	1.00	0.00	0.00	0.00	1.00	1.00	1.00	1.00	1.00
Output	**0.00**	**0.00**	**0.00**	**1.00**	**0.01**	**0.01**	**0.99**	**1.00**	**0.00**	**0.00**	**0.00**	**1.00**	**0.99**	**0.99**	**1.00**	**1.00**

Figure S3: **Logic design with Hill-type kinetics.** The figure caption is on the next page

Figure S3: **Logic design with Hill-type kinetics. (A) Reaction library with Hill kinetics.** To test GenAI-Net with reactions governed by multiple parameters, we construct a reaction library of 433 admissible reaction templates whose propensities follow Hill-type activation and repression forms (schematics). These templates include single-enzyme and multi-enzyme Hill functions that modulate effective production rates, yielding a larger action space for logic synthesis. **(B) 4-input logic circuit using Hill functions.** GenAI-Net starts from a *starter I/O CRN* (left) containing species $\mathbf{X_1}$–$\mathbf{X_6}$, with four external inputs u_i ($i = 1, \ldots, 4$) that modulate the production rates of $\mathbf{X_1}$–$\mathbf{X_4}$. All species are also assumed to dilute at a constant rate. The objective is to implement the target 4-input Boolean function shown above the panel. GenAI-Net appends up to **6 additional reactions** selected from the Hill library and optimizes a terminal-time truth-table loss identical to that in Fig. 7. The center plot shows output transients for the **top 30** topologically unique generated I/O CRNs (gray), with three representative solutions highlighted (colored; I/O CRNs 1, 15, and 30) and their corresponding topologies shown to the left. The network visualization (right) summarizes topological diversity across the 30 solutions. The truth-table panels (bottom) compare target outputs to simulated outputs across all 2^4 input combinations for the representative networks, demonstrating accurate multi-input logic computation with Hill-type kinetics. Note that, compared to Fig. 7, this Hill library is less flexible: only production reactions can be selected, whereas degradation reactions are not available. This constraint makes the task more demanding and motivates inclusion of an additional intermediate species ($\mathbf{X_5}$) to achieve high-precision logic behavior.

Oscillators with input-tunable frequency

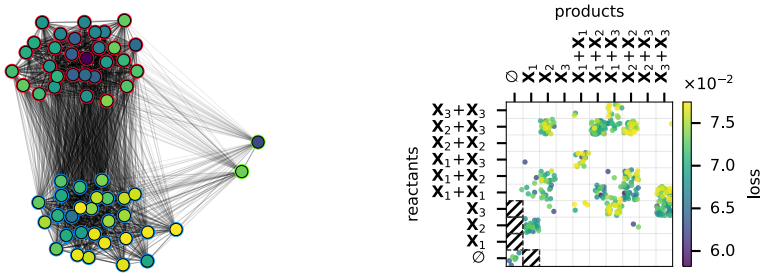


Figure S4: **Oscillators with input-tunable frequency.** The graph visualization and reactant–product incidence map summarizing topological diversity and reaction usage across the generated collections of Fig. 9.

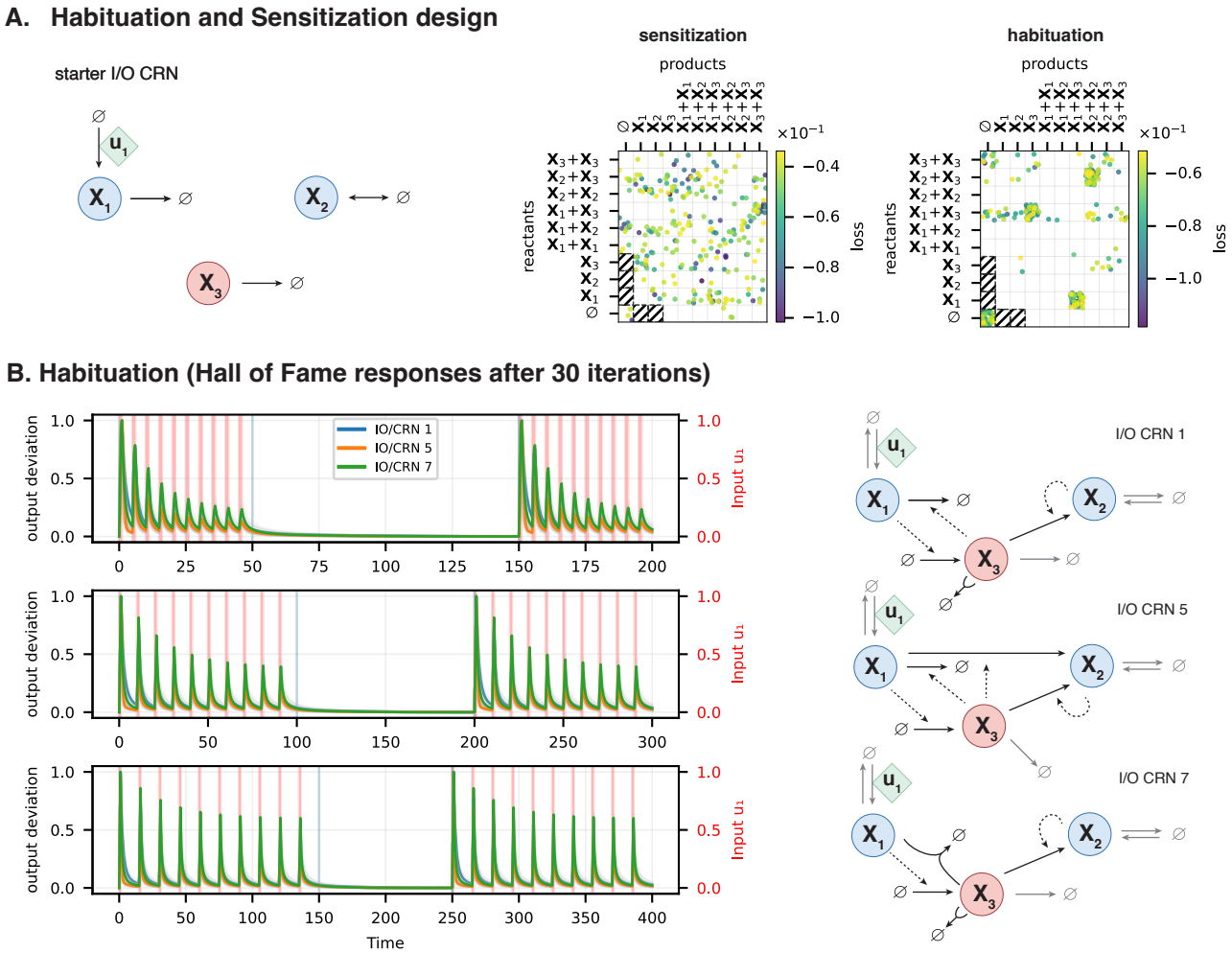


Figure S5: **Molecular habituation and sensitization. The figure caption is on the next page.**

Figure S5: **Design of biomolecular circuits exhibiting habituation and sensitization. (a) Problem setup and reaction signatures.** GenAI-Net starts from a *starter I/O CRN* with three species, an external input u_1, and a designated output species $\mathbf{X_3}$ (red). The design objective is to generate I/O CRNs whose output exhibits either *sensitization* (progressively increasing responses under repeated stimulation) or *habituation* (progressively decreasing responses), while preserving spontaneous recovery after an OFF gap (see Methods). The reactant–product incidence maps at right summarize reaction usage for the corresponding solution sets: each point denotes a reaction, and point color indicates the loss of I/O CRNs in which that reaction appears. **(b) Habituation.** After only 30 iterations, GenAI-Net identifies a hall-of-fame set of habituating I/O CRNs. The left panels show representative output trajectories for three highlighted networks (I/O CRNs 1, 5, and 7; colored curves), with the input pulse train u_1 overlaid in red. Across repeated pulses, the output peaks progressively attenuate, and after the OFF gap the response recovers before adapting again during the second pulse train, consistent with habituation and spontaneous recovery. Furthermore higher frequencies exhibit faster habituation. Gray trajectories indicate the remaining hall-of-fame solutions. The corresponding I/O CRN topologies are shown at right. These topologies show negative feedback structures, incoherent feedforward loop structure as reported in (*85, 86*), and buffering structures where $\mathbf{X_2}$ monitors the output and converts it to itself faster when the next pulse comes in. **(b) Sensitization.** After only 10 iterations, GenAI-Net likewise identifies a hall-of-fame set of sensitizing I/O CRNs. The left panels show representative trajectories for I/O CRNs 1, 4, and 7, again with the input pulse train overlaid in red. In contrast to (c), repeated stimulation now produces progressively larger output peaks, followed by recovery during the OFF gap and renewed sensitization under the second pulse train. The associated I/O CRN topologies are shown at right. Together, these results show that GenAI-Net can generate distinct circuit families that realize opposite forms of adaptive pulse-train response using the same overall design framework.

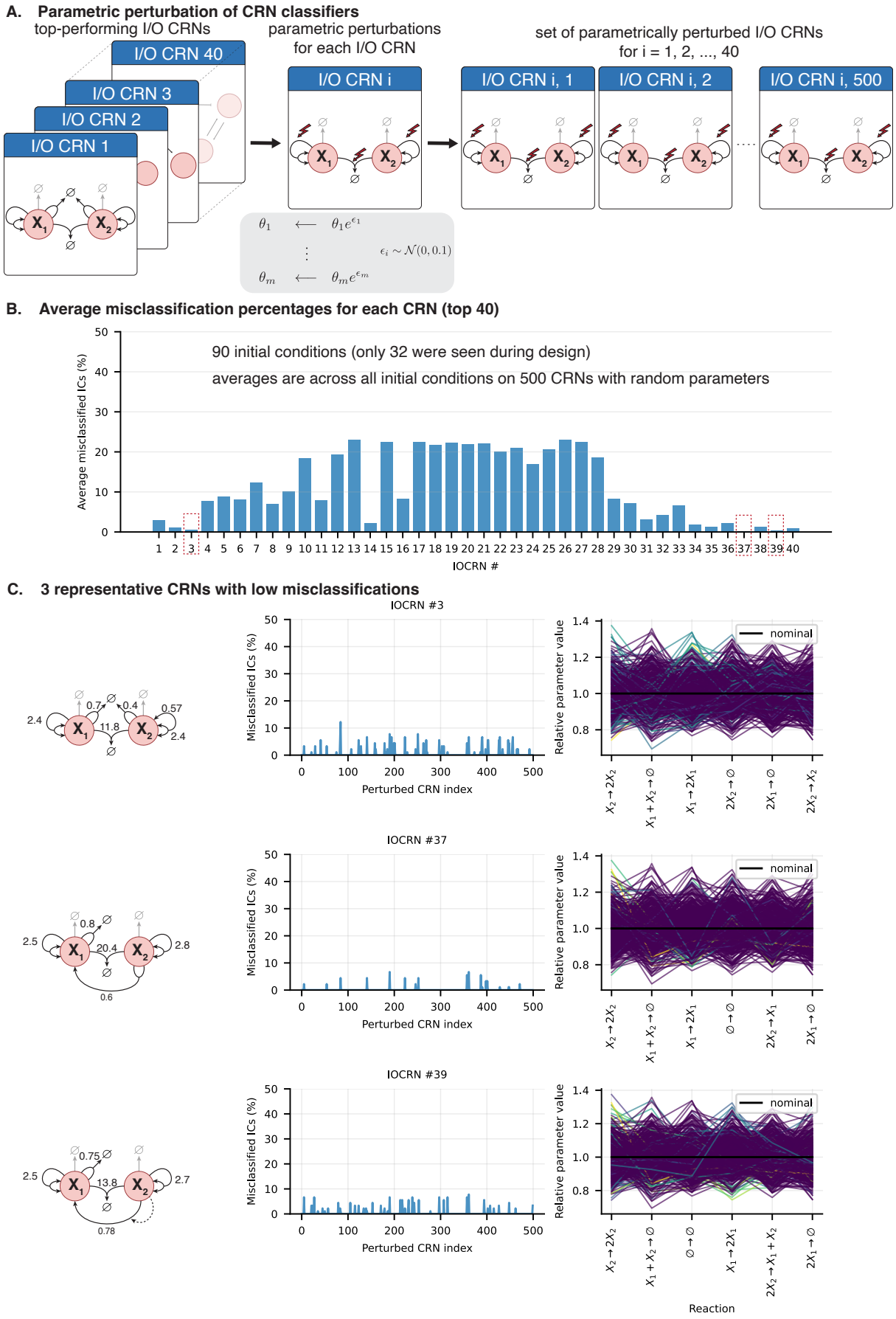
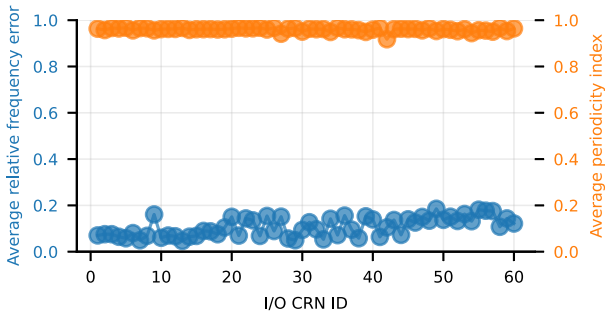


Figure S6: **Parametric robustness of initial-condition classifiers implemented by chemical reaction networks. The figure caption is on the next page.**

Figure S6: **Parametric robustness of initial-condition classifiers implemented by chemical reaction networks. (a) Robustness-analysis workflow.** The 40 top-performing, topologically distinct I/O CRNs were selected based on their nominal classification performance from Fig. 6. For each I/O CRN i, 500 independently perturbed variants were generated while preserving its reaction topology. Each reaction rate (excluding the starter reactions) was perturbed multiplicatively according to $\theta'_j = \theta_j e^{\epsilon_j}$, with independently sampled $\epsilon_j \sim \mathcal{N}(0, 0.1)$ and thus representing 10% relative parametric perturbations to all the parameters simultaneously. Lightning symbols indicate perturbed reaction-rate parameters. **(b) Mean percentage of misclassified initial conditions for each of the 40 nominal I/O CRNs after parameter perturbation.** Each bar is the average across the 500 independently perturbed variants and 90 initial conditions. The initial conditions form a grid as in Fig. 6. Only 32 of these conditions were used during CRN design; the remaining conditions test generalization beyond the design set. Low bars therefore indicate that the classifier's behavior is preserved despite random kinetic-parameter perturbations. Red dashed boxes identify the representative robust classifiers shown in panel c. **(c) Detailed robustness profiles for representative I/O CRNs 3, 37, and 39.** Left, reaction-network diagrams show the nominal topology and rounded kinetic parameters; gray reactions are the fixed template reactions, while black reactions were perturbed. Middle, percentage of the 90 initial conditions misclassified by each of the 500 perturbed variants. Right, corresponding perturbed kinetic parameters normalized by their nominal values; each polyline represents one perturbed CRN, and the horizontal black line denotes the nominal parameter values. Together, these examples demonstrate that several independently discovered CRN topologies retain accurate initial-condition classification across broad, simultaneous perturbations of their kinetic parameters even though this requirement was not explicitly encoded in the loss function.

A. Parametric perturbations: average frequency error and periodicity index (top 60)



B. 3 representative CRNs with low frequency errors

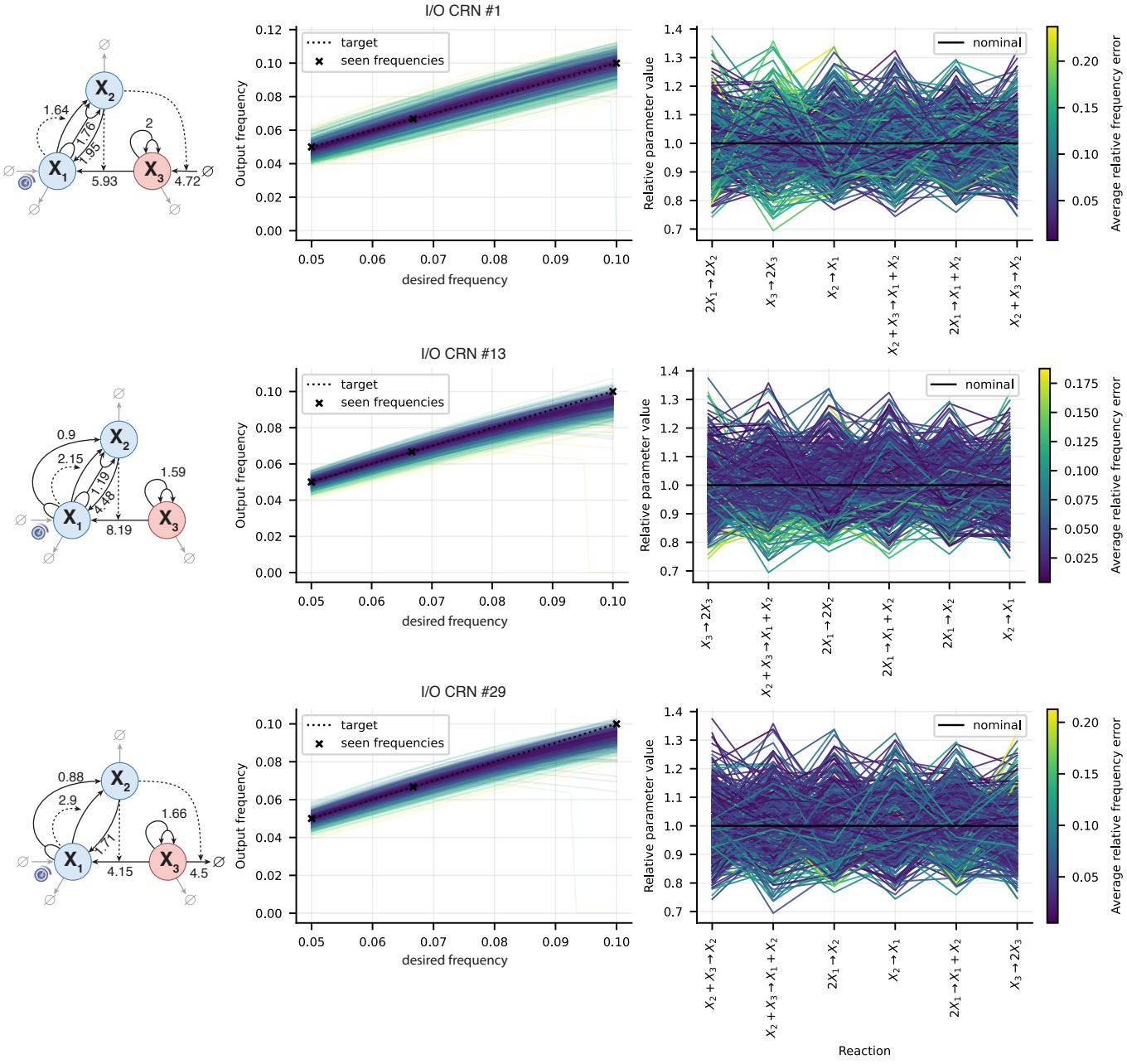


Figure S7: Parametric robustness of frequency-tracking chemical oscillators. The figure caption is on the next page.

Figure S7: **Parametric robustness of frequency-tracking chemical oscillators.** The parameter-perturbation procedure follows the workflow introduced in Fig. S6(a): The 60 top-performing, topologically distinct I/O CRNs were selected based on their nominal classification performance from Fig. 9. For each I/O CRN i, 500 independently perturbed variants were generated while preserving its reaction topology. Each reaction rate (excluding the starter reactions) was perturbed multiplicatively according to $\theta'_j = \theta_j e^{\epsilon_j}$, with independently sampled $\epsilon_j \sim \mathcal{N}(0, 0.1)$ and thus representing 10% relative parametric perturbations to all the parameters simultaneously. Each perturbed oscillator was evaluated at 100 desired frequencies (only 3 of which were seen during design) in the same way as in Fig. 9. **(a) Robustness of the 60 top-performing, topologically distinct frequency-tracking I/O CRNs.** Blue markers show the mean relative frequency error of the 500 perturbed variants of each nominal CRN, averaged across all tested desired frequencies. Orange markers show the corresponding mean periodicity index, where values approaching one indicate sustained periodic behavior. Thus, several frequency-tracking oscillators simultaneously exhibited maintained oscillations close to the desired input frequencies despite the parametric perturbations even though this was not encoded in the loss function. **(b) Detailed robustness profiles for three representative oscillators with low frequency errors: I/O CRNs 1, 13, and 29.** Left, reaction-network diagrams show the nominal CRN topologies and rounded kinetic parameters. Gray reactions denote the fixed template reactions, whereas black reactions were subjected to parameter perturbations; X_3 is the output species. Center, input-output frequency maps for the 500 perturbed variants. Each line represents one perturbed CRN evaluated over the full range of desired frequencies. The dotted diagonal indicates perfect frequency tracking, and crosses denote the three frequencies used during CRN design. Right, kinetic parameters of each perturbed variant normalized by their corresponding nominal values. Each polyline represents one perturbed CRN, and the horizontal black line denotes the nominal parameter vector. In the center and right plots, lines are colored by their average relative frequency error, linking changes in kinetic parameters to changes in frequency-tracking performance. Together, the results show that the selected CRNs generally retain sustained oscillations and accurate frequency tracking under simultaneous random perturbations of their kinetic parameters.

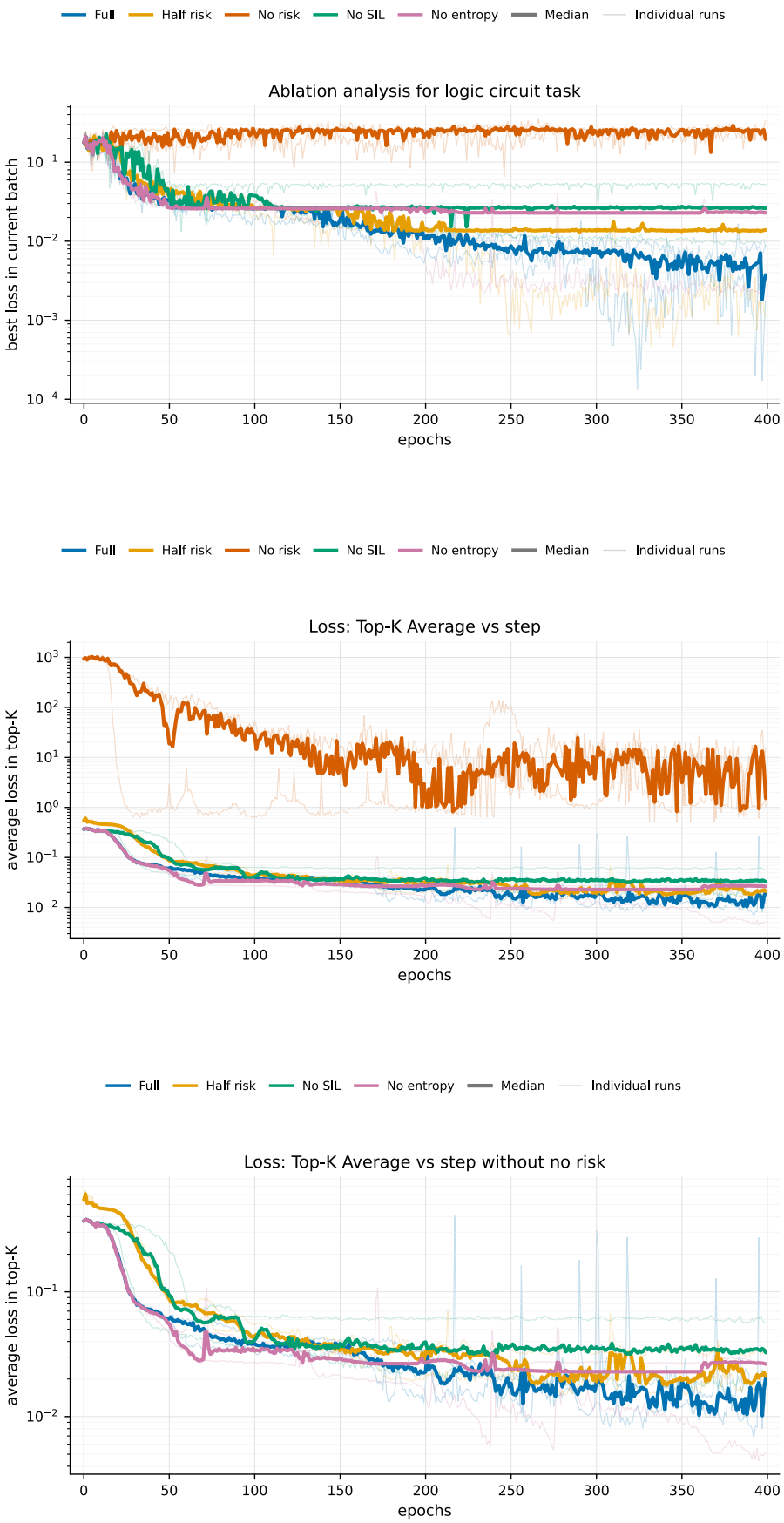


Figure S8: **Ablation analysis for the logic-circuit inference task. The figure caption is on the next page.**

Figure S8: **Ablation analysis for the logic-circuit inference task.** Training performance over 400 epochs for the complete GenAI-Net (Full) and four ablations: reducing the risk parameter from $r = 0.9$ to $r = 0.5$ (Half risk), removing risk seeking with $r = 0$ (No risk), disabling self-imitation learning (No SIL), and disabling entropy regularization (No entropy). The losses are plotted over the epochs; lower values indicate better logic-circuit performance. Thick colored curves show the median across three independent runs, while pale curves show individual runs (3 runs). All vertical axes use logarithmic scaling. Top: Best loss among the batch during each epoch. The full method achieves the lowest median batch-best loss by the end of training, whereas removing components substantially degrades performance and often gets stuck at local minima. Middle: Mean loss over the Top-K candidates, where $K = \lfloor 1280(1-r) \rfloor$: $K = 127$ for Full, No SIL, and No entropy; $K = 640$ for Half risk; and $K = 1280$ for No risk. The No risk condition remains markedly worse because its objective averages over the entire batch rather than emphasizing its best candidates. Bottom: The same Top-K metric with No risk omitted, revealing differences among the remaining conditions more clearly. The full method continues improving throughout training and ultimately attains the lowest median Top-K loss, while removing self-imitation learning or entropy regularization leads to earlier performance plateaus.